

KILONOVA: Preprocessing Folding-based SNARKs for Machine Executions

Tianyu Zheng¹, Shang Gao¹, Yu Guo², and Bin Xiao¹

¹ The Hong Kong Polytechnic University

² SECBIT Labs

Abstract. Succinct non-interactive arguments of knowledge have enabled efficient verification of complex computations, but practical applications such as zero-knowledge virtual machines face scalability challenges due to rapidly growing circuit sizes. While folding-based incrementally verifiable computation and proof-carrying data frameworks offer improved performance, existing constructions struggle to efficiently support non-uniform circuits. In this work, we present KILONOVA, a preprocessing recursive SNARK for universal machine execution. KILONOVA introduces a novel *holographic folding scheme* that enables efficient folding across multiple high-degree relations with non-uniform indices, achieving both asymptotic and concrete efficiency. Building on this, we construct a multi-predicate PCD system in which the prover maintains constant-size intermediate instances, thereby significantly reducing recursion overhead and memory usage. Finally, we instantiate KILONOVA as a preprocessing SNARK by compiling the PCD proof with corresponding polynomial commitment schemes. Theoretical analysis and experimental results show that KILONOVA achieves state-of-the-art performance in multi-predicate settings and therefore provides a scalable, practical foundation for zero-knowledge proofs in real-world applications.

1 Introduction

Succinct non-interactive arguments of knowledge (SNARKs) [7] have revolutionized the efficient verification of NP statements, particularly those represented as arithmetic or Boolean circuit satisfiability. However, practical computations are typically carried out as sequential instructions within computer programs, rather than as circuit evaluations. Consequently, it is both natural and desirable to construct efficient zero-knowledge proofs for real-world computations, enabling statements of the form: “when executing program P on public input x and private input w , the output is z ”. In the context of SNARKs, several systems have been proposed to prove the *correct machine execution* [5,10,25,1,37]. Due to its significant applications in *zero-knowledge virtual machines* (zkVMs) over the past few years, this research area has attracted considerable attention. It has been adapted to support diverse instruction sets, including EVM, RISC-V, and Wasm [36,29,1], and has played a pivotal role in enhancing the scalability of blockchain auditing [40].

However, as program complexity increases, the corresponding circuit size grows rapidly, resulting in significant overhead for both the prover’s runtime and memory. This inefficiency stems from the prevalent approach of “unrolling” the entire program execution trace into a single, universal arithmetic/boolean circuit, which is then processed by a cryptographic back-end. For instance, executing a typical zkEVM circuit with 2^{27} constraints using HyperPlonk requires approximately 25 minutes [17], highlighting the scalability challenges faced by current systems.

Incremental Computations. An alternative approach to proving machine execution is to decompose universal computation into multiple steps (*e.g.*, proving one instruction per step), and employ *Incrementally Verifiable Computation* (IVC) [45], or its generalization *Proof-Carrying Data* (PCD) [18]. In these frameworks, the cost of each proving step scales with the complexity of the executed instruction, rather than with the entire program trace. Canonical IVC/PCD constructions rely on recursive composition of general-purpose SNARKs, where at each step i , a SNARK verifies the proof generated at step $(i - 1)$ —a process known as *SNARK composition* [8]. This method requires embedding the full verification logic of the $(i - 1)$ -th SNARK within the proving circuit of the i -th SNARK (*i.e.*, the recursive circuit), incurring substantial overhead due to expensive cryptographic operations such as scalar multiplications and elliptic curve pairings [11]. Recent advances [11,15,9,35] have introduced more efficient constructions for IVC/PCD based on *folding schemes* or *accumulation schemes*³. Unlike SNARK composition, these approaches enable the prover to aggregate (fold) SNARK/NARK proofs and defer their verification until the end of the computation. This crucial distinction yields significant performance improvements, as verifying foldings requires far fewer cryptographic operations than verifying individual proofs.

Challenges. Folding schemes have thus far provided effective methods for constructing IVCs and PCDs. However, the performance of these primitives remains insufficient to meet the growing demands of real-world applications, particularly in the context of universal machine executions. The most significant limitation is that existing folding-based IVC/PCD constructions struggle to efficiently support non-uniform circuits. Current solutions primarily address non-uniformity by sacrificing succinctness; for example, SuperNova [32] requires the prover to maintain a list of all folded instances (*i.e.*, running instances), each with a distinct circuit structure (*i.e.*, the index). Alternatively, a folklore approach—implicitly referenced in Protostar and Protogalaxy [13,26]—suggests directly folding the commitments of distinct circuits. However, this method results in large recursive circuits due to the presence of non-native operations, which we later discuss as the *index folding problem* in Section 2.3. The discussion on other related work is presented in Appendix A.

³ We use these terms interchangeably, following [13].

Motivation. In this work, we aim to construct an efficient recursive SNARK for universal machine execution. To achieve this objective, we build on the index-folding technique proposed in Protostar [13] and address the following two key aspects. First, we seek to formalize the index folding technique as a holographic folding scheme, and to design a concrete protocol for folding multiple non-uniform inputs while mitigating cross-term issues. Second, we focus on further reducing the recursive overhead introduced by index folding and, consequently, construct an efficient multi-predicate PCD protocol with a constant number of running instances.

2 Technique Overview

In this work, we present KILONOVA, a preprocessing recursive SNARK for proving uniform machine execution. KILONOVA is constructed from two main components. First, we design a PCD system that supports multi-predicate statements within a single running instance, leveraging a novel holographic folding scheme as the core technique. Second, we develop a preprocessing SNARK based on SuperSpartan [43], in which all circuit matrices are preprocessed into commitments. The main contributions of this paper are summarized in Figure 1 with detailed descriptions provided in the subsequent parts. Before presenting our

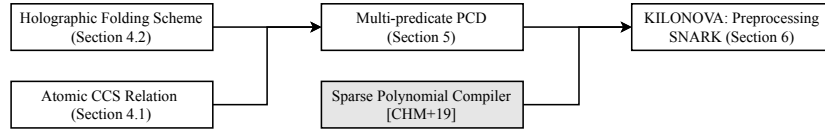


Fig. 1. Overview of KILONOVA contributions. We do not claim novelty for the technique in the gray box, which is adopted from Marlin [21].

main contributions, we first revisit HyperNova [34] in Section 2.1 to establish the necessary background. Subsequently, we provide a high-level overview of our core techniques: the holographic folding scheme (Section 2.2), the construction of multi-predicate PCD (Section 2.3), and the design of the final preprocessing SNARK (Section 2.4).

2.1 Recap of HyperNova

Since our work mainly develops on the idea in HyperNova [34] to present a folding scheme for CCS relations, we first review the necessary definitions, including Customizable Constraint System (CCS) relations, the SuperSpartan SNARK [43], and the Proof-Carrying Data (PCD) with generalization to the multi-predicate setting [23]. For notations, we denote an indexed relation $\mathcal{R}$, we define it as a set of tuples (i, x, w) including the index i , instance x , and witness w . More notations and preliminaries are introduced in Section 3.

Customizable Constraint Systems. Zero-knowledge proofs (ZKPs) rely on efficiently representing computations as mathematical statements. Over time, a variety of “intermediate representations” have been developed for this purpose, each offering distinct advantages. The Customizable Constraint System (CCS), introduced by Setty *et al.* [43], provides an elegant generalization of earlier popular representations such as R1CS [30], Plonkish arithmetic [28], and AIR [4], without incurring additional overhead. Concretely, we define the CCS index i to include: (i) t matrices $M_1, \dots, M_t \in \mathbb{F}^{m \times n}$, (ii) q multisets $S_1, \dots, S_q \subseteq \{1, \dots, t\}$, and (iii) q scalar constants $c_1, \dots, c_q \in \mathbb{F}$, where m, n, t, q are size bounds in public parameters. In this work, we focus on the *committed* CCS relation, where the instance x consists of (i) a commitment to the witness $w \in \mathbb{F}^{n-\ell-1}$, and (ii) a vector of public inputs $io \in \mathbb{F}^\ell$. The index-instance-witness tuple (i, x, w) satisfies relation $\mathcal{R}_{\text{CCS}}$ if the following holds:

$$\sum_{i \in [q]} c_i \cdot \bigcirc_{j \in S_i} (M_j \cdot \vec{z}) = \vec{0}, \quad (1)$$

where $\vec{z} = (io || 1 || w) \in \mathbb{F}^n$ is a concatenation, and $\bigcirc$ denotes Hadamard products for vectors. Clearly, the CCS can express high-degree constraints with $|S_i| > 2$.

SuperSpartan. Setty *et al.* proposed SuperSpartan [43] as a preprocessing SNARK for proving the CCS satisfiability. Let $B_k := \{0, 1\}^k$. Concretely, on input as a committed CCS instance, the prover and verifier first computes the multilinear extension (see Lemma 1) of all $\{M_j\}_{j \in [t]}$ and $\vec{z}$ as $\{\tilde{M}_j\}_{j \in [t]}$ and $\tilde{z}$. Then they interact to reduce Equation 1 to a sum-check statement as $\sum_{\vec{x} \in B_{\log m}} F(\vec{x}) = 0$ in terms of a multivariate polynomial $F(\vec{x}) :=$

$$\tilde{eq}(\vec{\alpha}, \vec{x}) \cdot \sum_{i \in [q]} c_i \prod_{j \in S_i} \sum_{\vec{y} \in B_{\log n}} \tilde{M}_j(\vec{x}, \vec{y}) \cdot \tilde{z}(\vec{y}), \quad (2)$$

where $\vec{\alpha}$ is randomly sampled from $\mathbb{F}^{\log m}$, and $\tilde{eq}(\vec{\alpha}, \vec{x})$ is the equality function defined in Lemma 1, which linearly combines m per-element constraints in Equation 1 into one.

To validate the sum-check statement, the prover and verifier run two rounds of sum-check protocols recursively on $\vec{x}$ and $\vec{y}$, respectively. After the first sum-check protocol, the verifier can check the final evaluation on $\vec{r}_x$ with the following claims from the prover:

$$\sum_{\vec{y} \in B_{\log n}} \tilde{M}_j(\vec{r}_x, \vec{y}) \cdot \tilde{z}(\vec{y}) = \sigma_j, \quad \forall j \in [t], \quad (3)$$

where $\vec{r}_x$ is the challenge vector generated during the first sum-check. These claims can be further checked by running t sum-check protocols in parallel, and the prover sends $t + 1$ claims for the verifier to check the final evaluation:

$$\tilde{M}_j(\vec{r}_x, \vec{r}_y) = \theta_j, \quad \forall j \in [t] \text{ and } \tilde{z}(\vec{r}_y) = \epsilon. \quad (4)$$

Proof Carrying Data. Kothapalli *et al.* propose an efficient multi-folding scheme for linearized CCS relation, which is derived from an “early stopping” version of SuperSpartan at Equation 3. Based on this, Zhou *et al.* construct a proof-carrying data [50], of which the notion is reviewed below. PCD [18] can be seen as a generalization of IVC to directed acyclic graphs (DAGs). This powerful paradigm enables a set of parties to carry out an arbitrarily long distributed computation where every step is accompanied by a proof of its correctness. At its core, a PCD system operates on a *transcript*, which is modeled as a directed acyclic graph T . Each vertex u in T (denoted $u \in V(T)$) is associated with some local data $z_{\text{loc}}^{(u)}$, and each edge e in T (denoted $e \in E(T)$) carries a message $z^{(e)}$. The overall output $o(T)$ of a transcript T is typically defined as the message $z^{(e)}$ of the lexicographically-first edge $e = (u, v)$ where v is a sink node (*i.e.*, has no outgoing edges). A vertex $u \in V(T)$ is deemed F -compliant for a given predicate F if, for all its outgoing edges $e = (u, v) \in E(T)$, the predicate $F(z^{(e)}, z_{\text{loc}}^{(u)}, \text{incoming messages})$ evaluates to true. Here, “incoming messages” refers to the messages $z^{(e_i)}$ carried by all edges e_i leading into u , which would be an empty set if u has no incoming edges. A transcript T itself is F -compliant if all of its vertices are F -compliant. Furthermore, PCD is said to be *multi-predicate* if its transcript is $\vec{F}$ -compliant, meaning each vertex is F -compliant for *some* predicate F chosen from a class of predicates $\vec{F}$, where the full definition is given in Appendix B.3.

2.2 Holographic Folding Scheme

Our first contribution is the introduction of a novel primitive, termed the *holographic folding scheme* (HFS), which enables efficient folding of multiple CCS tuples with non-uniform indices. As discussed in Section 1, our work builds upon ProtoStar and ProtoGalaxy [13,26], where tuples with different indices are directly folded. Unfortunately, these approaches suffer from efficiency limitations. Specifically, for CCS, given two tuples (i, x, w) and (i', x', w') , one can fold them into a new tuple (i^*, x^*, w^*) using a random challenge $\eta \leftarrow \mathbb{F}$ as follows:

$$i^* = i + \eta \cdot i', \quad x^* = x + \eta \cdot x', \quad w^* = w + \eta \cdot w'.$$

However, implementing this folding scheme presents two main challenges:

- To ensure that the folded tuple (i^*, x^*, w^*) satisfies the d -degree (relaxed) CCS relation, the prover must send a list of cross terms to the verifier, whose size grows exponentially with d .
- The verifier is required to perform an expensive check involving $O(tmn)$ field operations, as each index contains t matrices in $\mathbb{F}^{m \times n}$.

These challenges motivate the design of a new folding scheme that achieves both sublinear proof size and verifier complexity. We formally define this new primitive as a *holographic folding scheme* and describe two techniques that address the aforementioned challenges.

Definition. Although reducing verifier complexity proportional to the index size has seldom been studied, a related research problem has been studied in the preprocessing SNARK literature [42]. Typically, the verifier complexity of SNARK can be reduced to sublinear in the circuit size by partitioning the verification procedure into two phases: (i) an offline phase that produces a succinct summary (*i.e.*, encoding) for a given circuit, and (ii) an online phase that uses this summary to verify SNARK proofs attesting to the satisfiability of the circuit under different partial assignments to its input wires. This idea is further generalized by Chiesa *et al.* [21] to the notion of *holographic proofs*, where the verifier does not receive the circuit description as input, but instead makes a small number of queries to an encoding of the circuit. Motivated by this, we adopt the definition of holographic proofs for folding schemes, in which an indexer encodes the index into an oracle, thereby reducing the verifier’s complexity during folding. The formal definition of HFS is presented in Section 4.1.

Remark 1. We define HFS as a scheme in which the verifier is granted query access to the full index, as in [21]. While this property is not required when instantiated with a homomorphic PCS, we still retain it for generality, enabling our scheme to support non-homomorphic PCS such as code-based ones [16].

HFS Construction. We now provide an overview of the concrete construction of HFS, highlighting two key techniques.

Atomic CCS Relation. Recall the challenges that arise when directly folding two indexed instances. To address the first challenge, we explore the approach from HyperNova [34], which introduces a variant of the CCS relation called linearized committed CCS, derived from an “early stopping” version of SuperSpartan [43]. The main idea is that folding multiple (uniform) linearized CCS tuples satisfying the constraint in Equation 3 does not produce cross terms. Building on this observation, we additionally execute the second round of the sum-check protocol in SuperSpartan to further obtain an *atomic CCS* (ACCS) relation from the linearized CCS, as defined by the constraint in Equation 4. Consequently, ACCS tuples contain only independent linear claims on the index matrices $\tilde{M}_j$, thereby eliminating cross terms when folding multiple non-uniform tuples. A detailed definition of the ACCS relation is provided in Section 4.1.

Encoding the Index. To instantiate the indexer algorithm for HFS, we employ a homomorphic polynomial commitment scheme to encode the index matrices. Given an ACCS index $i = \{M_j\}_{j \in [t]}$, the indexer first computes the multilinear extension of each matrix, denoted as $\tilde{M}_j(\vec{X}, \vec{Y})$. For each $j \in [t]$, the indexer then commits to the polynomial as $C_{M,j} := \text{Commit}(\text{pp}, \tilde{M}_j(\vec{X}, \vec{Y}))$. As a result, the folding verifier can check the validity of the folded index i^* by computing its encoding as $\iota^* = \{C_{M,j}^*\}_{j \in [t]}$, where $C_{M,j}^* := C_{M,j} + \eta \cdot C'_{M,j}$ (using 2-folding for simplicity, where $C_{M,j}$ and $C'_{M,j}$ are encoded indices i and i' , respectively).

Leveraging both techniques, we construct a holographic folding scheme for multiple non-uniform CCS tuples. Informally speaking, the folding prover first

runs the SuperSpartan protocol to reduce each CCS tuple to an ACCS tuple. In addition to the input CCS tuples, the prover must also process the ACCS tuples from the previous node to facilitate folding. Specifically, folding μ CCS tuples with ν ACCS tuples requires an additional interactive protocol to “update” the ACCS tuples, which can be executed in parallel with the previous SuperSpartan protocol. As a result, we obtain a holographic, non-uniform, multi-folding scheme with proof size consisting of $O(\log n)$ field elements, independent of the number of input tuples, where n is the instance-witness length. The folding verifier’s complexity is dominated by $O(\log n)$ hash function evaluations (as instantiated random oracles) and $O((\mu + \nu)t)$ group operations. A step-by-step construction of the folding scheme is provided in Section 4.2.

2.3 Multi-Predicate PCD

As discussed in Section 1, existing solutions for multi-predicate IVC/PCD [15,32] require the prover to maintain a list of running instances, with the length scaling linearly to the number of “uninvoked” circuits. In Section 5, we present a novel construction of multi-predicate PCD based on the holographic folding scheme, in which the prover only needs to maintain a single running instance at each step. Before introducing the PCD construction, we first introduce an optimization for the folding verifier cost tailored to the PCD setting. We then describe the complete construction of multi-predicate PCD, followed by a theoretical comparison with existing IVC/PCD constructions.

Breaking the Barrier of Index Folding. Building on the holographic folding schemes described above, it is straightforward to derive a non-uniform PCD from previous constructions [15,51]. However, the recursive circuit remains large, as it must compute the index folding operations over matrix commitments, which involve $O((\mu + \nu)t)$ “non-native” group operations.⁴

To address this issue, we introduce an optimization in Section 5.1 that reduces from $O((\mu + \nu)t)$ “non-native” group operations in the recursive circuit to $O((\mu + \nu)|\vec{F}|)$ field operations, where $|\vec{F}|$ denotes the size of the predicate set. We illustrate our idea as follows.

Let $\vec{i}_F$ denote the predicate index set in $\vec{F}$. Since each CCS index corresponds to a predicate in $\vec{F}$, folding CCS indices is essentially equivalent to folding the predicates. Accordingly, the index of the running instance in PCD, *i.e.*, the folded ACCS index, can always be expressed as a random linear combination of the predefined predicate indices in $\vec{i}_F$ using a random scalar vector $\mathbf{r} \in \mathbb{F}^{|\vec{F}|}$. For example, for the index part of an ACCS tuple output by the folding scheme, we have $i^* = \sum_{k=1}^{|\vec{F}|} r_k \cdot \vec{i}_F[k]$, where $\vec{i}_F[k]$ is the k -th index in $\vec{i}_F$.

At each step of PCD, the correctness of index folding can be efficiently checked with a random linear combination for scalar vectors. For instance, computing $i + \eta \cdot i'$ is equivalent to $(\sum_{k=1}^{|\vec{F}|} r_k \cdot \vec{i}_F[k]) + \eta \cdot (\sum_{k=1}^{|\vec{F}|} r'_k \cdot \vec{i}_F[k]) = \sum_{k=1}^{|\vec{F}|} (r_k +$

⁴ Even with the use of a 2-cycle of curves, this cost remains prohibitive.

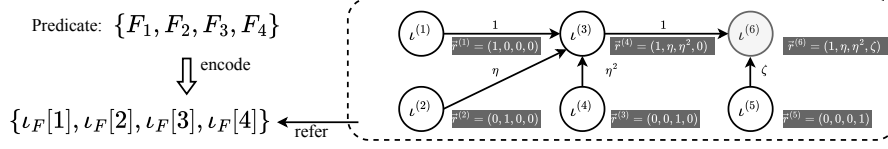


Fig. 2. An example for illustrating the index folding in PCD. The sink node contains a valid folded index $\iota^{(6)} := \langle \vec{r}^{(6)}, \vec{\iota}_F \rangle$, which implies the validity of each $C_{M,j}$ in $\iota^{(6)}$.

$\eta \cdot \mathbf{r}'_k) \cdot \vec{\mathbf{i}}_F[k]$. Thus, instead of folding ι 's (which would cost $O((\mu + \nu)t)$ group operations) in the recursive circuit, we only need to fold the scalar vectors $\mathbf{r}$'s, requiring just $O((\mu + \nu)|\vec{F}|)$ field operations. Moreover, any query for $\mathbf{i}$ can be obtained by querying all indices in $\vec{\mathbf{i}}_F$, and combining the results with its corresponding $\mathbf{r}$, *i.e.*, $\mathbf{i} = \sum_{k=1}^{|\vec{F}|} \mathbf{r}_k \cdot \vec{\mathbf{i}}_F[k]$.

As a tradeoff, the PCD verifier must perform $O(t|\vec{F}|)$ group operations locally. The general idea is illustrated in Figure 2, where $\vec{\iota}_F$ denotes the encoded predicate index set for $\vec{F}$.

PCD Construction. The construction of multi-predicate PCD is detailed in Section 5.2. In general, for each vertex in the graph, the algorithm takes as input μ CCS tuples and ν ACCS tuples from the output of previous vertices. The vertex then runs the PCD proving algorithm with the following two steps:

1. Execute the folding prover algorithm to fold all input tuples into a single folded ACCS tuple.
2. Instantiate a CCS tuple from a circuit that encodes (i) the compliance predicate and (ii) the folding verification logic.

Finally, the prover outputs a new pair of ACCS and CCS tuples for the next vertex. The main insight of our construction is summarized in Theorem 1 below.

Theorem 1. *Assume a constant-round, public-coin holographic folding scheme for μ degree- d CCS tuples $\{(i, \mathbf{x}, \mathbf{w})\}$ and ν ACCS tuples $\{(i', \mathbf{x}', \mathbf{w}')\}$ with instance and witness lengths as a constant n . Then, a multi-predicate PCD for constant-size compliance predicates $\vec{F}$ can be constructed based on non-interactive holographic folding schemes in the Random Oracle Model (ROM). Moreover, the recursion overhead is dominated by $O(\log n)$ hash function evaluations, $O(\mu + \nu)$ group operations, and $O(d \log n + (\mu + \nu)|\vec{F}|)$ field operations.*

Theoretical Comparisons. We compare the theoretical performance of KILONOVA with existing IVC/PCD constructions in Table 1. For simplicity, we assume $\nu = O(\mu)$ and omit the ν term. As shown, KILONOVA is the only scheme that supports both multiple predicates and practical multi-folding. In terms of

Table 1. Comparisons between existing IVC/PCD. The “Deg” refers to the maximum degree supported by the constraint systems, “MP” indicates multiple predicate, “MF” indicates multiple folding. The row “KILONova” refers to the naive construction while “KILONova (opt.)” refers to Construction 2 with optimizations in 5.1. For parameters, RO denotes the Random Oracle query, n is the witness length and d is the highest degree of supported constraints ($d = 2$ for R1CS), μ is the number of input instance, t is the matrix number in CCS and $\vec{F}$ is the predicate set.

	Deg	MP	MF	Recursive overhead	Proof size
Nova [35]	2	✗	✗	$O(1)$ RO, $O(1)$ $\mathbb{G}$	$O(1)$ $\mathbb{G}$, $O(n)$ $\mathbb{F}$
SuperNova [32]	2	✓	✗	$O(1)$ RO, $O(1)$ $\mathbb{G}$	$O(\vec{F})$ $\mathbb{G}$, $O(\vec{F} \cdot n)$ $\mathbb{F}$
HyperNova [34]	d	✗	✓	$O(\log n)$ RO, $O(\mu)$ $\mathbb{G}$	$O(1)$ $\mathbb{G}$, $O(t + n)$ $\mathbb{F}$
BCLMS21 [14]	2	✗	✓	$O(1)$ RO, $O(\mu)$ $\mathbb{G}$	$O(\vec{F})$ $\mathbb{G}$, $O(\vec{F} \cdot n)$ $\mathbb{F}$
Protostar [13]	d	✗	✗	$O(1)$ RO, $O(d + t)$ $\mathbb{G}$	$O(d)$ $\mathbb{G}$, $O(t + n)$ $\mathbb{F}$
Protogalaxy [26]	d	✗	✓	$O(1)$ RO, $O(\mu \cdot (d + t))$ $\mathbb{G}$	$O(d)$ $\mathbb{G}$, $O(t + n)$ $\mathbb{F}$
KILONova	d	✓	✓	$O(\log n)$ RO, $O(\mu \cdot t)$ $\mathbb{G}$	$O(t)$ $\mathbb{G}$, $O(t + n)$ $\mathbb{F}$
KILONova (opt.)	d	✓	✓	$O(\log n)$ RO, $O(\mu)$ $\mathbb{G}$	$O(1)$ $\mathbb{G}$, $O(t + n + \vec{F})$ $\mathbb{F}$

recursion overhead, the optimized KILONova achieves $O(\mu)$ group operations,⁵ which is significantly better than the $O(\mu(d + t))$ group operations required by Protogalaxy, and is comparable to the performance of HyperNova, albeit under single-predicate settings. Regarding proof size, KILONova matches the proof size of HyperNova, except for an additional cost incurred by transmitting a scalar vector of size $O(|\vec{F}|)$ field elements. Although this introduces a linear term in the predicate size (*i.e.*, the number of uninvoked circuits), the concrete performance remains substantially better than previous solutions, including Protogalaxy [26] and SuperNova [32], as demonstrated in Section 7.

2.4 Preprocessing SNARK for Universal Machine Execution

In this part, we mainly describe how to instantiate KILONova as a preprocessing SNARK for proving universal machine execution. We present a preprocessing zkSNARK for machine execution traces, leveraging a sparse polynomial commitment scheme and the HyperKZG protocol [48]. Our approach encodes the machine description as machine commitments during preprocessing and enables efficient verification against them.

Compiling Sparse Polynomial Oracles. To mitigate the prohibitive quadratic complexity associated with encoding dense matrices, we introduce a sparse polynomial commitment scheme by generalizing the approach of [21] to the multivariate setting. This adaptation enables the prover’s computational complexity to scale linearly with respect to the number of non-zero entries in the matrix. Specifically, the prover reduces a single evaluation claim on the $\Omega(n^2)$ -size polynomial

⁵ This cost corresponds to the folding of $\text{Commit}(\text{pp}, \mathbf{w})$, which is independent of the (encoded) index and is required across all multi-folding schemes.

$\tilde{M}$ to three sub-claims involving $\Omega(N)$ -size polynomials $\tilde{\text{row}}, \tilde{\text{col}}, \tilde{\text{val}}$, where $\tilde{M}$ denotes the multilinear extension of the matrix $M \in \mathbb{F}^{n \times n}$ containing at most N non-zero entries. This decomposition significantly improves efficiency while preserving soundness and succinctness.

Preprocessing zkSNARK for ACCS. For atomic CCS relations, we construct a succinct zero-knowledge proof system by masking witness information and aggregating evaluation proofs. In particular, zero-knowledge for the atomic CCS relation is achieved by appending a dummy tuple to the final multi-folding scheme in the PCD protocol, thereby ensuring that the witness is masked by a randomly sampled value in the dummy tuple. We employ the HyperKZG scheme for polynomial commitments, which enables efficient verification with only two pairing operations and logarithmic group scalar multiplications. Moreover, HyperKZG supports the aggregation of multiple evaluation proofs, further reducing the overall proof size. Our preprocessing zkSNARK attains the following efficiency guarantees: the indexer operates in $O(N)$ time, the prover in $O(N \log N)$ time, the proof size is $O(\log n)$, and the verifier runs in $O(\log n)$ time, where n denotes the matrix dimension and N the number of non-zero entries. This framework provides a scalable and practical solution for succinctly proving machine execution traces, making it suitable for real-world applications.

Practical Comparisons We implement KILONOVA in Rust with the Arkworks library, and benchmark its performance against SuperNova [32], HyperNova [34], and Protogalaxy [26] on a consumer-grade laptop, as shown in Figure 3. For prover time (Figure 3(a)), KILONOVA matches HyperNova in single-predicate (uniform) settings and significantly outperforms Protogalaxy, indicating that the second sum-check introduces minimal overhead. In multi-predicate (non-uniform) settings, HyperNova’s proving time would increase (above KILONOVA) if index-folding were conducted directly; since this is not part of their design, we omit its performance in this scenario. KILONOVA also achieves smaller proof sizes than SuperNova in non-uniform settings (Figure 3(b)), due to the constant number of running instances enabled by index folding. Additionally, KILONOVA exhibits lower recursive overhead compared to Protogalaxy (Figure 3(c)), owing to optimizations in index folding. Further evaluations are provided in Section 7.

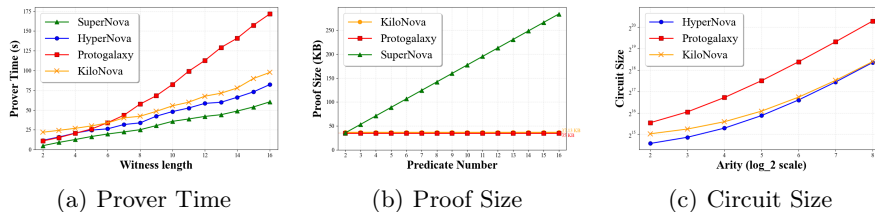


Fig. 3. Performance Comparison between KILONOVA and other works [32,34,26].

3 Preliminaries

3.1 Notations

In this paper, we use λ to denote the security parameter. Accordingly, $\text{negl}(\lambda)$ denotes an unspecified function that is negligible in λ . We denote by $[n]$ the set $\{1, \dots, n\} \subseteq \mathbb{N}$. Let $\mathbb{F}$ denote a finite field, *e.g.*, a prime field for a large prime p . The vector is denoted as $\vec{a} \in \mathbb{F}^n$ with elements $a_1, \dots, a_n \in \mathbb{F}$. $\vec{a}[i]$ is also used to denote the i -th element of $\vec{a}$ when the element is not specified with a concrete value. To represent a set, we use $\{a_i\}_{i=1}^n$ as a short-hand for $\{a_1, \dots, a_n\}$. From here on, we let $B_\mu := \{0, 1\}^\mu \subseteq \mathbb{F}^\mu$ be the boolean hypercube. For a finite set S , let $x \leftarrow \$ S$ denote sampling x from S uniformly at random. We use “PPT algorithms” to refer to “Probabilistic Polynomial Time” algorithms, and “DPT algorithms” to refer to “Deterministic Polynomial Time” algorithms.

Polynomials. We recall some basic definitions for polynomials from [42] as follows. Let $f(\cdot) : \mathbb{F}^n \rightarrow \mathbb{F}$ be a *multivariate polynomial* with n input elements over $\mathbb{F}$; its total degree d is defined as the maximum degree over all monomials in $f(\cdot)$. Moreover, the degree of a polynomial in a specified variable x_i is the maximum exponent that x_i takes in any of the monomials in $f(\cdot)$. Particularly, a multivariate polynomial is *multilinear* if the degree of the polynomial in each variable is at most one. To be consistent with notations of vectors, we use $f(\vec{x})$ to denote the polynomial $f(\cdot)$ with the specified input variable as vector $\vec{x}$. Next, we state the lemmas used in our paper.

Lemma 1 (Multilinear extensions [44]). *Let $f(\cdot) : B_n \rightarrow \mathbb{F}$ be a function that maps n -bit elements into an element of $\mathbb{F}$. The multilinear extension of $f(\cdot)$ is a unique multilinear n -variate polynomial $\tilde{f}(\cdot) : \mathbb{F}^n \rightarrow \mathbb{F}$ such that $\tilde{f}(\vec{x}) = f(\vec{x})$ for all $\vec{x} \in B_n$, which can be computed as $\tilde{f}(\vec{x}) = \sum_{\vec{e} \in B_n} f(\vec{e}) \cdot \tilde{eq}(\vec{x}, \vec{e})$, where $\tilde{eq}(\vec{x}, \vec{e}) := \prod_{i=1}^n (x_i \cdot e_i + (1 - x_i) \cdot (1 - e_i))$.*

Relations. An indexed relation $\mathcal{R}$ for a non-deterministic polynomial time (NP) is a set of triples (i, x, w) , where i is the index, x is the instance, and w is the witness; The corresponding indexed language $\mathcal{L}(\mathcal{R})$ is the set of pairs (i, x) for which there exists a witness w such that $(i, x, w) \in \mathcal{R}$. For example, the indexed R1CS relation for satisfiable arithmetic circuits consists of triples where i represents the matrices describing the arithmetic circuit, x is the public assignment to a subset of the input wires, and w is the witness assignment to the remaining wires that satisfies the circuit.

3.2 Sum-check Protocol

The sum-check protocol is an interactive proof proposed by Lund *et al.* [38]. Assume $\tilde{f}(\cdot) : \mathbb{F}^n \rightarrow \mathbb{F}$ as an n -variate low-degree polynomial with the maximum

degree of d for each variable. The prover wants to convince the verifier of the following claim:

$$\text{sum} = \sum_{x_1 \in \{0,1\}} \sum_{x_2 \in \{0,1\}} \cdots \sum_{x_n \in \{0,1\}} \tilde{f}(x_1, \dots, x_n). \quad (5)$$

To do this, the prover and verifier participate in an n -round interactive proof. At $i \in [n]$ round, the prover fixes the polynomial variable x_i at r_i randomly sampled by the verifier, and the verifier checks correspondingly. By applying the Fiat-Shamir transform, the above process can be written into a non-interactive version. In this work, we denote the non-interactive sum-check protocol as a tuple of algorithms $\text{SC} = (\mathcal{G}, \mathcal{P}, \mathcal{V})$,

- the generator $\mathcal{G}$ inputs a unary 1^λ denoted security length and outputs public parameters pp .
- the prover $\mathcal{P}$ inputs public parameters pp , a target polynomial $\tilde{f}$, variable length n , maximum degree d and claimed sum sum , outputs a transcript ts and the challenge vector $\vec{r}$.
- the verifier $\mathcal{V}$ inputs public parameters pp , a transcript ts , variable length n , maximum degree d , and claimed sum as sum , outputs 0 (reject) or 1 (accept).

At the final step, the verifier outputs a claim about the evaluation $f(\vec{r})$, *i.e.*, $c \leftarrow \Pi_{\text{sc}}(f, n, d, \text{sum}, \vec{r})$. If $c = f(\vec{r})$ holds, then the verifier is convinced of the claim about the sum of $f(\cdot)$ in Equation (5). According to previous work [38,42], the sum-check protocol satisfies both completeness and soundness properties, and its communication cost takes $O(n \cdot d)$ elements of $\mathbb{F}$.

3.3 Polynomial Commitment Scheme

We adopt the definition of polynomial commitment schemes (PCS) by Boneh [9].

Definition 1 (Polynomial Commitment Scheme). *A polynomial commitment scheme PCS for multilinear polynomials is defined as a tuple of four protocols $\text{PC} = (\text{Gen}, \text{Commit}, \text{Open}, \text{Eval})$:*

- $\text{Gen}(1^\lambda, \ell) \rightarrow \text{pp}$: *takes as input ℓ (the number of variables in a multilinear polynomial); produces public parameters pp .*
- $\text{Commit}(\text{pp}, \tilde{f}) \rightarrow C$: *takes as input an ℓ -variate multilinear polynomial $\tilde{f} : \mathbb{F}^\ell \rightarrow \mathbb{F}$; produces a commitment C .*
- $\text{Open}(\text{pp}, C, \tilde{f}) \rightarrow b$: *verifies the opening of commitment C to the ℓ -variate multilinear polynomial $\tilde{f}$; outputs $b \in \{0, 1\}$.*
- $\text{Eval}(\text{pp}, C, \vec{x}, y, \ell; \tilde{f}) \rightarrow \pi$: *takes as input the polynomial commitment C , the evaluation point $(\vec{x}, y)$ and the ℓ -variate multilinear polynomial $\tilde{f}$, outputs an evaluation proof π .*
- $\text{Check}(\text{pp}, C, \vec{x}, y, \ell, \pi) \rightarrow b$: *takes as the polynomial commitment C , the evaluation point $(\vec{x}, y)$, the variable length ℓ , and the evaluation proof π , outputs $b = 1$ if π attests that, for $\vec{x}$, the ℓ -variable polynomial $\tilde{f}$ committed in C evaluates to y at $\vec{x}$.*

Definition 2. A polynomial commitment scheme for multilinear polynomials $\text{PC} = (\text{Setup}, \text{Commit}, \text{Open}, \text{Eval}, \text{Check})$ is *additively homomorphic* if for all ℓ and pp produced from $\text{Setup}(1^\lambda, \ell)$, and for any $\tilde{f}_1, \tilde{f}_2 : \mathbb{F}^\ell \rightarrow \mathbb{F}$,

$$\text{Commit}(\text{pp}, \tilde{f}_1) + \text{Commit}(\text{pp}, \tilde{f}_2) = \text{Commit}(\text{pp}, \tilde{f}_1 + \tilde{f}_2).$$

A PC is an extractable polynomial commitment scheme for multilinear polynomials over a finite field $\mathbb{F}$ if it satisfies completeness, binding, and knowledge soundness properties as defined in Appendix B.1. We also mention a useful lemma in the security proofs.

Lemma 2 (Schwartz-Zippel lemma [41]). Assume $\tilde{f}(\cdot) : \mathbb{F}^n \rightarrow \mathbb{F}$ is a non-zero n -variate polynomial of degree at most d . Then on any finite set $S \subseteq \mathbb{F}$,

$$\Pr_{\vec{x} \leftarrow S^n} [\tilde{f}(\vec{x}) = 0] \leq d/|S|,$$

where $\vec{x}$ is a randomly sampled vector from S^n and $|S|$ denotes the size of S .

3.4 Proof Systems

Interactive Proofs. Let $\mathcal{R} := \{(i, x, w)\}$ be an NP relation. An interactive proof for $\mathcal{R}$ enables a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ that a statement x has a corresponding witness w under index i . Typically, a $(2\mu - 1)$ -move protocol consists of μ messages from the prover and $\mu - 1$ challenges from the verifier, exchanged in an interactive manner. If all messages from $\mathcal{V}$ are generated uniformly at random and independently of the prover's messages, the protocol is referred to as a *public-coin* protocol. Such protocols can be transformed into non-interactive ones via the Fiat-Shamir heuristic [27]. For security definitions, the interactive proof should satisfy *completeness* and *knowledge soundness*. To facilitate security analysis, the notion of *special soundness* is often employed, as it implies knowledge soundness according to ACK21 [3]. Informally, an interactive proof satisfies k -special soundness if there exists an efficient extractor that, given the index-instance pair (i, x) and k accepting transcripts with the same first message but distinct challenges (in a prescribed manner), can output a valid witness w .

Definition 3 (k -special soundness [2]). Π provides k -special soundness, if there is an effective PPT extraction algorithm $\mathcal{E}$ that is capable of extracting the witness w given (i, x) and any k accepting transcripts $\vec{T}$. Specifically, for all PPT adversaries $\mathcal{A}$, it holds that

$$\Pr [\text{pp} \leftarrow \mathcal{G}(1^\lambda), (i, x, \vec{T}) \leftarrow \mathcal{A}(\text{pp}), w \leftarrow \mathcal{E}(\text{pp}, i, x, \vec{T}) : (i, x, w) \in \mathcal{R}] \approx 1.$$

General-purpose NARKs. We define a non-interactive argument of knowledge NARK for an indexed NP relation $\mathcal{R}$ as a tuple of algorithms $(\mathcal{G}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ below, each with access to the random oracle RO. NARK is secure in the random oracle model if the following properties hold:

Definition 4 (Completeness). NARK is complete if for every (unbounded) adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{c} (i, x, w) \notin \mathcal{R}(\text{pp}) \\ \vee \\ \text{NARK. } \mathcal{V}^{\text{RO}}(\text{vk}, x, \pi) = 1 \end{array} \middle| \begin{array}{c} \text{RO} \leftarrow \mathcal{U}(\lambda) \\ \text{pp} \leftarrow \text{NARK. } \mathcal{G}^{\text{RO}}(1^\lambda) \\ (i, x, w) \leftarrow \mathcal{A}^{\text{RO}}(\text{pp}) \\ (\text{pk}, \text{vk}) \leftarrow \text{NARK. } \mathcal{I}^{\text{RO}}(\text{pp}, i) \\ \pi \leftarrow \text{NARK. } \mathcal{P}^{\text{RO}}(\text{pk}, x, w) \end{array} \right] = 1.$$

Definition 5 (Knowledge soundness). NARK is knowledge sound with respect to auxiliary input distribution Ψ if for every expected polynomial-time adversary $\tilde{\mathcal{P}}$, there exists an expected polynomial-time extractor Ext such that for every set Z ,

$$\Pr \left[\begin{array}{c} (\text{pp}, \text{ai}, \vec{i}, \vec{x}, \text{ao}) \in Z \wedge \forall j \in [\ell], \\ \text{NARK. } \mathcal{V}^{\text{RO}}(\text{vk}_j, x_j, w_j) = 1 \end{array} \middle| \begin{array}{c} \text{RO} \leftarrow \mathcal{U}(\lambda) \\ \text{pp} \leftarrow \text{NARK. } \mathcal{G}(1^\lambda) \\ \text{ai} \leftarrow \Psi(\text{pp}) \\ (\vec{i}, \vec{x}, \vec{\pi}, \text{ao}) \leftarrow \tilde{\mathcal{P}}^{\text{RO}}(\text{pp}, \text{ai}) \\ \forall j \in [\ell], (\text{pk}_j, \text{vk}_j) \leftarrow \text{NARK. } \mathcal{I}^{\text{RO}}(\text{pp}, i_j) \end{array} \right]$$

$$\leq \Pr \left[\begin{array}{c} (\text{pp}, \text{ai}, \vec{i}, \vec{x}, \text{ao}) \in Z \\ \wedge \\ \forall j \in [\ell], (i_j, x_j, w_j) \in \mathcal{R}(\text{pp}) \end{array} \middle| \begin{array}{c} \text{RO} \leftarrow \mathcal{U}(\lambda) \\ \text{pp} \leftarrow \text{NARK. } \mathcal{G}(1^\lambda) \\ \text{ai} \leftarrow \Psi(\text{pp}) \\ (\vec{i}, \vec{x}, \vec{w}, \text{ao}) \leftarrow \text{Ext}^{\tilde{\mathcal{P}}}(\text{pp}, \text{ai}) \end{array} \right] + \text{negl}(n)(\lambda).$$

We omit the random oracle if the context is clear. For more details, we encourage the readers to refer to the definitions in [22]. Moreover, the NARK is said to be *succinct*, if the communication complexity (and sometimes the work of the verifier) is sublinear in the witness size are called succinct. A *preprocessing* SNARK allows anyone can publicly preprocess a given index (*e.g.*, a circuit) in an offline phase, in order to avoid incurring costs related to the index in (any number of) subsequent online phases that check different instances.

4 Holographic Folding Schemes

In this section, we introduce a novel holographic folding scheme that supports folding multiple non-uniform instance-witness pairs. We begin by presenting two new definitions: the *holographic folding scheme* and the *atomic CCS relation* in Section 4.1. Subsequently, we provide a concrete instantiation of the holographic non-uniform multi-folding scheme for CCS relations (referred to as the holographic folding scheme for brevity) in Section 4.2.

4.1 New Definitions

Holographic Folding Schemes. In HyperNova [34], the authors proposed a multi-folding scheme for relations $\mathcal{R}$ and $\mathcal{R}'$ parameterized by sizes μ and ν . This

scheme enables a prover and a verifier to jointly fold μ instance-witness pairs in $\mathcal{R}$ and ν pairs in $\mathcal{R}'$ into a single pair in $\mathcal{R}'$, under the constraint that all pairs share the *same* index.

In this work, we generalize the multi-folding scheme to the non-uniform setting, where the prover and verifier fold instance-witness pairs associated with *different* indices. Concretely, given a pair of relations $(\mathcal{R}, \mathcal{R}')$ and size parameters μ, ν , the prover receives as input $(\vec{i}, \vec{x}, \vec{w}) \in \mathcal{R}^\mu$ and $(\vec{i}', \vec{x}', \vec{w}') \in (\mathcal{R}')^\nu$ (i.e., μ -many $\mathcal{R}$ tuples and ν -many $\mathcal{R}'$ tuples), while the verifier receives $(\vec{i}, \vec{x})$ and $(\vec{i}', \vec{x}')$. By executing an interactive protocol, both parties agree on a folded index-instance pair (i^*, x^*) , and the prover privately computes a folded witness w^* such that $(i^*, x^*, w^*) \in \mathcal{R}'$.

A central challenge in designing such a folding scheme lies in deriving the new index i^* on the verifier's side. Since the indices often consist of a large number of elements (for example, t matrices of size $\mathbb{F}^{m \times n}$ in CCS), directly computing their linear combinations can be prohibitively inefficient, as discussed in Section 1.

To address the aforementioned challenge, we draw inspiration from the concept of holographic proofs [21, 22], where the verifier accesses only succinct encodings of the indices rather than the full index itself. Analogously, we incorporate an indexer to encode the indices, allowing the folding scheme to fold the encoded indices rather than the indices themselves. This enables the verifier to efficiently check the validity of i^* using the compact encoding. We formalize folding schemes satisfying these properties as a *holographic non-uniform multi-folding scheme* in Definition 6, which we refer to as the *holographic folding scheme* (HFS) for brevity.

Definition 6 (Holographic folding schemes). Consider relations $\mathcal{R}$ and $\mathcal{R}'$, with size parameters μ and ν respectively, both sharing the same public parameters pp . A holographic folding scheme for $(\mathcal{R}, \mathcal{R}', \mu, \nu)$ is a tuple consisting of a PPT generator algorithm $\mathcal{G}$, a DPT indexer algorithm $\mathcal{I}$, and a pair of PPT algorithms $\mathcal{P}$ and $\mathcal{V}$ denoting the prover and the verifier, respectively, with syntaxes as follows:

- $\mathcal{G}(1^\lambda) \rightarrow \text{pp}$: on input security parameter λ , samples public parameters pp .
- $\mathcal{I}(\text{pp}, \vec{i}, \vec{i}') \rightarrow (\text{fpk}, \text{fvk})$: on input pp , index vectors $\vec{i}, \vec{i}'$ of size μ and ν , outputs a prover key fpk and a verifier key fvk . Typically, fpk includes $\vec{i}, \vec{i}'$, and fvk includes short encodings of all indices as $\vec{\iota}, \vec{\iota}'$.
- $\mathcal{P}(\text{fpk}, (\vec{x}, \vec{w}), (\vec{x}', \vec{w}')) \rightarrow (i^*, x^*, w^*, \pi)$: on input fpk , a vector of μ instance-witness pairs $(\vec{x}, \vec{w})$ in $\mathcal{R}$, and a vector of ν instance-witness pairs $(\vec{x}', \vec{w}')$ in $\mathcal{R}'$, the prover outputs a folded tuple as (i^*, x^*, w^*) with a proof π .
- $\mathcal{V}(\text{fvk}, \vec{x}, \vec{x}', i^*, x^*, \pi) \rightarrow b$: on input fvk , a vector of μ instances $\vec{x}$, a vector of ν instances $\vec{x}'$, a folded index-instance pair (i^*, x^*) with proof π , the verifier outputs either $b = 1$ (accept) or $b = 0$ (reject).

Remark 2. Note that the verifier in Definition 6 is also granted oracle access to the index according to the definition of holographic schemes. While this capability is not necessarily required for our construction, we still cover it in our definition for potential optimizations as mentioned in Quasar [49].

Denote an interactive holographic folding scheme as a tuple of algorithms $\text{HFS} = (\mathcal{G}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ above. Accordingly, an HFS should retain perfect completeness and computational knowledge soundness as defined in Appendix B.4. We additionally require the verifier complexity to be sublinear to the index size $|\mathbf{i}|$. To represent the relation between an index and its encoding, we abuse the denotation $\iota = I(\mathbf{i})$.

Atomic CCS Relations. As mentioned in Section 1, existing folding schemes encounter the “cross term” problem when dealing with multiple inputs with different structures. A practical solution is to reduce the degree of relations satisfied by the input tuples, such that applying the random linear combinations on the tuples will not produce too many cross terms. However, even when considering the folding scheme in [34], folding linearized committed CCS tuples in non-uniform settings still produces cross terms. Motivated by the SuperSpartan IOP [43], we address this by continue with the second round of the sum-check protocol from SuperSpartan, which yields Equation (3). Note that Equation (4) contains claims on $\tilde{M}_j(\vec{r}_x, \vec{r}_y)$ for all $j \in [t]$ and $\tilde{z}(\vec{r}_y)$ separately, each with degree at most 1. As a result, folding these claims produces no cross terms, even in non-uniform settings. We denote this new variant of CCS as *Atomic CCS* and formalize it in the following definition.

Definition 7 (Atomic CCS). Let $\text{PCS} = (\text{Gen}, \text{Commit}, \text{Open}, \text{Eval})$ denote an additively-homomorphic polynomial commitment scheme for multilinear polynomials over a finite field $\mathbb{F}$. Denote the public parameters of size bounds as $m, n, N, \ell, t \in \mathbb{N}$ where $n = 2 \cdot (\ell + 1)$ and $\text{pp} \leftarrow \text{Gen}(1^\lambda, \log n - 1)$. The Atomic Customizable Constraint System (ACCS) relation $\mathcal{R}_{\text{ACCS}}$ is defined as.

- an ACCS index $\mathbf{i}$ consists of a sequence of sparse multilinear polynomials in $\log m + \log n$ variables $\{\tilde{M}_j\}_{j \in [t]}$ such that they evaluate to a non-zero value in at most $N = \Omega(m)$ locations over the boolean hypercube $B_{\log m} \times B_{\log n}$;
 - an ACCS instance $\mathbf{x}$ contains $(C, u, \mathbf{io}, \vec{r}_x, \vec{r}_y, v_1, \dots, v_t, v_z)$ where $u \in \mathbb{F}$, $\mathbf{io} \in \mathbb{F}^\ell$, $\vec{r}_x \in \mathbb{F}^{\log m}$, $\vec{r}_y \in \mathbb{F}^{\log n}$, $v_z \in \mathbb{F}$, $v_j \in \mathbb{F}$ for all $j \in [t]$, and C is a commitment to a witness $\mathbf{w}$;
 - an ACCS witness $\mathbf{w}$ is a vector in $\mathbb{F}^{n-\ell-1}$.
- An ACCS index-instance pair $(\mathbf{i}, \mathbf{x})$ is satisfied by $\mathbf{w}$ if $v_z = \tilde{z}(\vec{r}_y)$, $\text{Commit}(\text{pp}, \tilde{\mathbf{w}}) = C$ and if for all $j \in [t]$, the equation $v_j = \tilde{M}_j(\vec{r}_x, \vec{r}_y)$ holds, where $\tilde{M}_j(\vec{x}, \vec{y})$ is a $(\log m + \log n)$ -variate multilinear polynomial, $\tilde{z}(\vec{y})$ is an $\log n$ -variate multilinear extension of $\tilde{z} := (\mathbf{w} \parallel u \parallel \mathbf{io})$ on $\vec{y} \in B_{\log n}$.

4.2 Full Scheme

Based on the ACCS relations, we present the concrete construction for the holographic folding scheme as per Definition 6, which folds μ ACCS tuples and ν committed CCS tuples with different indices into one ACCS tuple.

- For CCS tuples, the folding prover and verifier first aggregate all inputs into one and run an “early-stopping” SuperSpartan [43] to reduce the aggregated CCS tuple into an ACCS tuple.

- For input ACCS tuples from the previous step, the folding prover can not directly fold them with the newly derived ACCS tuple at this step because these two tuples consist of *different* challenges $\vec{r}_x, \vec{r}_y$. To address this, we use a well-known technique [46,42] to “update” the random vectors of the ACCS tuple with a sum-check protocol, yielding new random vectors.

For simplicity, we first present the step-by-step description of a 2-folding scheme in Construction 1, *i.e.*, $\mu = \nu = 1$, which can be easily extended to the multi-folding case by computing the linear combinations of CCS tuples and ACCS tuples respectively.

Construction 1 (Holographic folding scheme for CCS). *Let $\text{PCS} = (\text{Gen}, \text{Commit}, \text{Eval}, \text{Verify})$ denote an additively homomorphic polynomial commitment scheme for multilinear polynomials. Let $\text{SC} = (\mathcal{G}, \mathcal{P}, \mathcal{V})$ denote a tuple of algorithms for a non-interactive sum-check protocol. The presented folding scheme only involves two tuples $(i, x, w) \in \mathcal{R}_{\text{CCS}}$, $(i', x', w') \in \mathcal{R}_{\text{ACCS}}$, where*

$$i = (\{\tilde{M}_j\}_{j \in [t]}, \{S_i\}_{i \in [q]}, \{c_i\}_{i \in [q]}), \quad x = (C, \text{io}),$$

$$i' = (\{\tilde{M}'_j\}_{j \in [t]}), \quad x' = (C', u', \text{io}', \vec{r}_x', \vec{r}_y', \{v'_j\}_{j \in [t]}, v'_z).$$

We highlight the variables related to the atomic CCS relation $\mathcal{R}_{\text{ACCS}}$ in *blue* fonts. Without loss of generality, we assume i and i' share the same size bounds m, n, N, ℓ, t , but contain different multilinear polynomials, $\{\tilde{M}_j\}_{j \in [t]}$ and $\{\tilde{M}'_j\}_{j \in [t]}$. Let $s_x = \log m$, $s_y = \log n$, $z = (w \| 1 \| \text{io})$, and $z' = (w' \| u \| \text{io}')$. We construct a non-uniform folding scheme as a non-interactive protocol consisting of a tuple of algorithms $\text{NIFS} = (\mathcal{G}, \mathcal{I}, \mathcal{P}, \mathcal{V})$, all of which have access to the same random oracle RO . The description of each algorithm is given below.

NIFS. $\mathcal{G}(1^\lambda \rightarrow \text{pp}) :$

- 1 : Sample size bounds $m, n, N, \ell, t, q, d \in \mathbb{N}$ with $n = 2 \cdot (\ell + 1)$.
- 2 : $\text{pp}_{\text{PCS}} \leftarrow \text{PCS.Gen}(1^\lambda, \log n - 1)$.
- 3 : $\text{pp}_{\text{SC}} \leftarrow \text{SC.}\mathcal{G}(1^\lambda)$.
- 4 : Output $\text{pp} = (m, n, N, \ell, t, q, d, \text{pp}_{\text{PCS}}, \text{pp}_{\text{SC}})$.

NIFS. $\mathcal{I}(\text{pp}, i, i') \rightarrow (\text{fpk}, \text{fvk}) :$

- 1 : Parse $\{\tilde{M}_j\}_{j \in [t]}$ from i .
- 2 : Parse $\{\tilde{M}'_j\}_{j \in [t]}$ from i' .
- 3 : Encode $\{\tilde{M}_j\}_{j \in [t]}$ as $\iota = \vec{C}_M$, where $\vec{C}_M[j] := \text{PCS.Commit}(\text{pp}_{\text{PCS}}; \tilde{M}_j)$.
- 4 : Encode $\{\tilde{M}'_j\}_{j \in [t]}$ as $\iota' = \vec{C}'_M$, where $\vec{C}'_M[j] := \text{PCS.Commit}(\text{pp}_{\text{PCS}}; \tilde{M}'_j)$.
- 5 : $\text{fpk} \leftarrow (\text{pp}, i, i')$.
- 6 : $\text{fvk} \leftarrow (\text{pp}, \iota, \iota')$.
- 7 : Output (fpk, fvk) .

NIFS. $\mathcal{P}^{\text{RO}}(\text{fpk}, (\mathbf{x}, \mathbf{w}), (\mathbf{x}', \mathbf{w}')) \rightarrow (\mathbf{i}^*, \mathbf{x}^*, \mathbf{w}^*, \pi) :$

- 1 : Parse $\mathbf{i}, \mathbf{i}'$ from fpk .
- 2 : Query $\text{RO}(\mathbf{i}, \mathbf{i}', \mathbf{x}, \mathbf{x}')$ and get $\gamma \in \mathbb{F}, \vec{\alpha} \in \mathbb{F}^{s_x}$.
- 3 : Parse $\{\tilde{M}_j\}_{j \in [t]}, \{S_i\}_{i \in [q]}, \{c_i\}_{i \in [q]}$ from $\mathbf{i}$, and $\{\tilde{M}'_j\}_{j \in [t]}$ from $\mathbf{i}'$.
- 4 : Compute $\tilde{z}(\vec{y}), \tilde{z}'(\vec{y})$ from $\tilde{z}, \tilde{z}'$.
- 5 : Run $(\mathbf{ts}_x, \vec{r}_x^*) \leftarrow \text{SC}(\text{pp}_{\text{sc}}, f, s_x, d+1, \text{sum}_x)$, where

$$\text{sum}_x := \sum_{j \in [t]} \gamma^j \cdot v_j,$$

$$f(\vec{x}) := \left(\sum_{j \in [t]} \gamma^j \cdot L_j(\vec{x}) \right) + \gamma^t \cdot Q(\vec{x}),$$

$$L_j(\vec{x}) := \tilde{e}q(\vec{r}_x^*, \vec{x}) \cdot \left(\sum_{\vec{y} \in B_{s_y}} \tilde{e}q(\vec{r}_y^*, \vec{y}) \cdot \tilde{M}'_j(\vec{x}, \vec{y}) \right), \quad j \in [t],$$

$$Q(\vec{x}) := \tilde{e}q(\vec{\alpha}, \vec{x}) \cdot \left(\sum_{i \in [q]} c_i \cdot \prod_{j \in S_i} \left(\sum_{\vec{y} \in B_{s_y}} \tilde{M}_j(\vec{x}, \vec{y}) \cdot \tilde{z}(\vec{y}) \right) \right).$$
- 6 : Compute claims $(\{\sigma_j\}_{j \in [t]}, \{\sigma'_j\}_{j \in [t]})$ as

$$\sigma_j = \sum_{\vec{y} \in B_{s_y}} \tilde{M}_j(\vec{r}_x^*, \vec{y}) \cdot \tilde{z}(\vec{y}), \quad \forall j \in [t],$$

$$\sigma'_j = \sum_{\vec{y} \in B_{s_y}} \tilde{e}q(\vec{r}_y^*, \vec{y}) \cdot \tilde{M}'_j(\vec{r}_x^*, \vec{y}), \quad \forall j \in [t].$$
- 7 : Query $\text{RO}(\mathbf{ts}_x, (\{\sigma_j\}_{j \in [t]}, \{\sigma'_j\}_{j \in [t]}))$ and get $\delta \in \mathbb{F}$.
- 8 : Run $(\mathbf{ts}_y, \vec{r}_y^*) \leftarrow \text{SC}(\text{pp}_{\text{sc}}, g, s_y, 2, \text{sum}_y)$, where

$$\text{sum}_y := \sum_{j \in [t]} \delta^j \cdot \sigma_j + \delta^{t+1} \cdot v_z + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot \sigma'_j,$$

$$g(\vec{y}) := \sum_{j \in [t]} \delta^j \cdot R_j(\vec{y}) + \delta^{t+1} \cdot S(\vec{y}) + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot T_j(\vec{y}),$$

$$R_j(\vec{y}) = \tilde{M}_j(\vec{r}_x^*, \vec{y}) \cdot \tilde{z}'(\vec{y}), \quad \forall j \in [t],$$

$$S(\vec{y}) = \tilde{e}q(\vec{r}_y^*, \vec{y}) \cdot \tilde{z}(\vec{y}),$$

$$T_j(\vec{y}) = \tilde{e}q(\vec{r}_y^*, \vec{y}) \cdot \tilde{M}'_j(\vec{r}_x^*, \vec{y}), \quad \forall j \in [t].$$
- 9 : Compute claims $(\epsilon, \epsilon', \{\theta_j\}_{j \in [t]}, \{\theta'_j\}_{j \in [t]})$ as

$$\epsilon = \tilde{z}(\vec{r}_y^*), \quad \epsilon' = \tilde{z}'(\vec{r}_y^*),$$

$$\theta_j = \tilde{M}_j(\vec{r}_x^*, \vec{r}_y^*), \quad \theta'_j = \tilde{M}'_j(\vec{r}_x^*, \vec{r}_y^*), \quad \forall j \in [t].$$
- 10 : Query $\text{RO}(\mathbf{ts}_y, (\epsilon, \epsilon', \{\theta_j\}_{j \in [t]}, \{\theta'_j\}_{j \in [t]}))$ and get $\eta \in \mathbb{F}$.
- 11 : Compute the folded triples $(\mathbf{i}^*, \mathbf{x}^*, \mathbf{w}^*)$ as

$$\mathbf{i}^* = \mathbf{i} + \eta \cdot \mathbf{i}' = \{M_j + \eta \cdot M'_j\}_{j \in [t]},$$

$$\mathbf{w}^* = \mathbf{w} + \eta \cdot \mathbf{w}',$$

$$\mathbf{x}^* = \mathbf{x} + \eta \cdot \mathbf{x}' = (C + \eta \cdot C', \text{io} + \eta \cdot \text{io}', 1 + \eta \cdot u, \{\theta_j + \eta \cdot \theta'_j\}_{j \in [t]}, \epsilon + \eta \cdot \epsilon').$$
- 12 : Output the folded pair $(\mathbf{i}^*, \mathbf{x}^*)$ and proof π as

$$(\gamma, \vec{\alpha}, \mathbf{ts}_x, (\{\sigma_j\}_{j \in [t]}, \{\sigma'_j\}_{j \in [t]}), \delta, \mathbf{ts}_y, (\epsilon, \epsilon', \{\theta_j\}_{j \in [t]}, \{\theta'_j\}_{j \in [t]}), \eta).$$

NIFS. $\mathcal{V}^{\text{RO}}(\text{fvk}, \mathbf{x}, \mathbf{x}', \mathbf{i}^*, \mathbf{x}^*, \pi) \rightarrow b :$

- 1 : Parse ι, ι' from fvk , including $\vec{C}_M, \vec{C}'_M$.
- 2 : Parse π as $(\gamma, \vec{\alpha}, \mathbf{ts}_x, (\{\sigma_j\}_{j \in [t]}, \{\sigma'_j\}_{j \in [t]}), \delta, \mathbf{ts}_y, (\epsilon, \epsilon', \{\theta_j\}_{j \in [t]}, \{\theta'_j\}_{j \in [t]}), \eta)$.
- 3 : Query $\text{RO}(\iota, \iota', \mathbf{x}, \mathbf{x}')$ and get $\gamma \in \mathbb{F}, \vec{\alpha} \in \mathbb{F}^{s_x}$.

```

4 : Run  $b_x \leftarrow \text{SC}.\mathcal{V}(\text{pp}_{\text{sc}}, \text{ts}_x, s_x, d+1, \text{sum}_x)$ , abort if  $b_x = 0$ .
5 : Ensure claims  $(\{\sigma_j\}_{j \in [t]}, \{\sigma'_j\}_{j \in [t]})$  as
      
$$c_x = \left( \sum_{j \in [t]} \gamma^j \cdot e_1 \cdot \sigma'_j \right) + \left( \gamma^t \cdot e_2 \cdot \sum_{i \in [q]} c_i \cdot \prod_{j \in S_i} \sigma_j \right).$$

6 : Query  $\text{RO}(\text{ts}_x, (\{\sigma_j\}_{j \in [t]}, \{\sigma'_j\}_{j \in [t]}))$  and get  $\delta \in \mathbb{F}$ .
7 : Run  $b_y \leftarrow \text{SC}.\mathcal{V}(\text{pp}_{\text{sc}}, \text{ts}_y, s_y, d+1, \text{sum}_y)$ , fail if  $b_y = 0$ .
8 : Ensure claims  $(\epsilon, \epsilon', \{\theta_j\}_{j \in [t]}, \{\theta'_j\}_{j \in [t]})$  as
      
$$c_y = \sum_{j \in [t]} \delta^j \cdot e_3 \cdot \theta'_j + \delta^{t+1} \cdot e_3 \cdot \epsilon' + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot \theta_j \cdot \epsilon.$$

9 : Query  $\text{RO}(\text{ts}_y, (\epsilon, \epsilon', \{\theta_j\}_{j \in [t]}, \{\theta'_j\}_{j \in [t]}))$  and get  $\eta \in \mathbb{F}$ .
10 : Check if  $\iota^* = \iota + \eta \cdot \iota'$ .
11 : Check the validity of  $x^*$  as per step 11 in NIFS.  $\mathcal{P}$ .
12 : Output the  $b = 1$  if all checks above pass, else  $b = 0$ .

```

Efficiency. Denote the random oracle for folding schemes and sum-check protocol as RO_{fs} and RO_{sc} . In Construction 1, the *folding scheme prover*

- asks $\log m + \log n$ queries to RO_{sc} and 4 queries to RO_{fs} ;
- computes sum-check protocols with $O(\mu(N + tm) + \nu(N + tm + qmd \log^2 d))$ $\mathbb{F}$ -ops for all μ atomic CCS tuples and ν committed CCS tuples according to SuperSpartan [43];
- performs $O(\mu + \nu)$ $\mathbb{G}$ -ops to combine $\{C^{(k)}\}_{k=1}^{\mu+\nu}$;
- performs $O((\mu + \nu)t)$ $\mathbb{G}$ -ops to combine matrix commitments among all ACCS or committed CCS tuples;
- performs $O((\mu + \nu)n)$ $\mathbb{F}$ -ops to fold witness vectors.

The *folding scheme verifier*

- asks $\log m + \log n$ queries to RO_{sc} and 4 queries to RO_{fs} ;
- checks sum-check protocols with $O(d \log m + \log n)$ $\mathbb{F}$ -ops.
- checks claims with $O(qt)$ $\mathbb{F}$ -ops;
- performs $O(\mu + \nu)$ $\mathbb{G}$ -ops to combine $\{C^{(k)}\}_{k=1}^{\mu+\nu}$;
- performs $O((\mu + \nu)t)$ $\mathbb{G}$ -ops to combine matrix commitments among all ACCS or committed CCS tuples.

Theorem 2. (*Security for Holographic Folding Scheme*). *Construction 1 is a non-interactive holographic folding scheme that satisfies perfect completeness and computational knowledge soundness under the random oracle model. If the random oracle is instantiated with a cryptographic hash function, the obtained non-interactive folding scheme is heuristically secure in the standard model under the Fiat-Shamir heuristic.*

Proof. (Proof sketch of Theorem 2). We left the formal proof, including the discussion on completeness, to Appendix C.1. For knowledge soundness, the general proof idea is to first demonstrate that the interactive holographic folding scheme HFS satisfies special soundness. Then, according to the Fiat-Shamir heuristic, the transformed NIFS preserves knowledge soundness in the random oracle model. To illustrate, we argue that given two accepting transcripts with the same initial

messages but different challenges η_1, η_2 from the malicious prover, an extractor can recover two satisfying witnesses $\mathbf{w}, \mathbf{w}'$ to the original $(i, \mathbf{x})$ and $(i', \mathbf{x}')$ efficiently. Specifically, the extractor first computes the folded witnesses $\mathbf{w}, \mathbf{w}'$ from the two transcripts by solving the linear system

$$\mathbf{w}_1^* = \mathbf{w} + \eta_1 \cdot \mathbf{w}', \quad \mathbf{w}_2^* = \mathbf{w} + \eta_2 \cdot \mathbf{w}',$$

To demonstrate that the extracted witnesses satisfy the CCS and ACCS relations with overwhelming probability, we rely on the soundness of the sum-check protocol. Given that the evaluation claims $(\{\sigma_j\}_{j \in [t]}, \{\sigma'_j\}_{j \in [t]}), (\epsilon, \epsilon', \{\theta_j\}_{j \in [t]}, \{\theta'_j\}_{j \in [t]})$ passes the verification algorithm **NIFS**. $\mathcal{V}$, the following equation holds with overwhelming probability according to the soundness of the sum-check protocol:

$$\sum_{j \in [t]} \gamma^j \cdot v_j = \sum_{\vec{x} \in B_{s_x}} \left(\left(\sum_{j \in [t]} \gamma^j \cdot L_j(\vec{x}) \right) + \gamma^{t+1} \cdot Q(\vec{x}) \right).$$

Then, ensured by the Schwartz-Zippel lemma, we have that

$$v_j = \sum_{\vec{x} \in B_{s_x}} L_j(\vec{x}) \quad \wedge \quad 0 = \sum_{\vec{x} \in B_{s_x}} Q(\vec{x}).$$

i.e., the extracted witnesses $\mathbf{w}, \mathbf{w}'$ satisfy the CCS and ACCS constraints in i, i' . Finally, we argue that the malicious prover either knows the satisfying witnesses $\mathbf{w}, \mathbf{w}'$ or breaks the binding property of the polynomial commitment scheme PCS with negligible probability, which concludes the proof. $\square$

5 Multi-Predicate PCD

In this section, we describe the practical construction of PCD based on the holographic folding scheme in Section 4. To begin with, we discuss the existing efficiency problem incurred by the folding operations for CCS matrix commitments. We introduce a practical solution in Section 5.1 by proposing a preprocessing NARK for PCD. Based on this, we detail the PCD construction in Section 5.2 and analyze its performance.

5.1 Preprocessing NARK for PCD

In the holographic folding scheme (HFS), the prover and verifier agree on a folded index encoding ι^* after their interaction. Even though the cost of folding the encoded indices is independent of the index size (*i.e.*, the size of the matrices), this approach introduces two significant challenges for constructing PCD.

Efficiency Challenge. The first challenge concerns the substantial recursive overhead introduced by index folding operations. Although the folding verifier computes only a random linear combination over succinct matrix commitments (*i.e.*,

the index encodings) at step 10 of NIFS. $\mathcal{V}$, we emphasize that this computation remains nontrivial, especially in the context of high-arity PCD. Specifically, given μ ACCS and ν CCS tuples—each associated with distinct indices at every node—the recursive circuit must implement the folding verifier algorithm, incurring at least $O((\mu + \nu)t)$ scalar multiplications over the group. Since a single scalar multiplication typically requires approximately 1,200 multiplication gates on standard elliptic curves [33], this recursive overhead can quickly dominate the prover’s computational cost.

System Design Challenge. Beyond efficiency, the construction also faces a system design challenge: at each node, the folding prover generates a new folded index i^* that does not belong to the original predicate set. Consequently, the current node must rerun the indexer algorithm to update the corresponding fpk and fvk for the subsequent node. This process (i) increases communication costs between nodes, as it necessitates transmitting matrices of size $O(tmn)$ over $\mathbb{F}$, and (ii) raises memory requirements, since the folding prover must maintain both the indices of the predefined predicate set $\vec{F}$ and an additional list of folded indices proportional to the arity. In practical deployments, where high arity is often adopted to enable parallelism, this memory overhead can become a critical bottleneck for PCD prover performance.

Preprocessing NARK from Folded CCS. We begin by presenting a preprocessing NARK construction based on committed CCS, and subsequently illustrate an optimization technique. Consider a committed CCS index-instance pair (i, x) satisfied by a witness w , where

$$i = (\{M_j\}_{j \in [t]}, \{S_i\}_{i \in [q]}, \{c_i\}_{i \in [q]}) \quad \wedge \quad x = (C, \text{io}).$$

We describe a straightforward NARK construction, with the indexer, prover, and verifier operating as follows:

- **NARK Indexer:** Computes $\iota = \{C_{M,j}\}_{j \in [t]}$ from i , and outputs

$$\text{pk} := i \quad \wedge \quad \text{vk} := (\iota, \{S_i\}_{i \in [q]}, \{c_i\}_{i \in [q]}).$$

- **NARK Prover:** Executes two rounds of the sum-check protocol to reduce the CCS constraint in Equation 2 to the ACCS constraint in Equation 4. The prover then opens the witness together with the sum-check transcripts ts_x, ts_y and evaluation claims $\{\theta_j\}_{j \in [t]}, \epsilon$.
- **NARK Verifier:** Given $\text{vk} = \iota$, queries the matrix commitments $C_{M,j}$ in ι to obtain the evaluations $\tilde{M}_j(\vec{r}_x, \vec{r}_y)$ for $j \in [t]$. The verifier checks the validity of ts_x, ts_y and verifies the following constraints:

$$\tilde{z}(\vec{r}_y) = \epsilon \quad \wedge \quad C = \text{Commit}(\text{pp}; \tilde{w}) \quad \wedge \quad \theta_j = \tilde{M}_j(\vec{r}_x, \vec{r}_y), \quad (6)$$

where $\tilde{z}$ denotes the multilinear extension of $\vec{z} := (\text{io} \| u \| w)$.

For the predicate set $\vec{F}$, let $\vec{i}_F$ denote the corresponding set of indices. Observe that the folded index i^* of the ACCS tuple (output by the folding prover) is essentially an RLC of the index set $\vec{i}_F$. Specifically, assuming the index encoding algorithm is homomorphic, for any honestly generated (i^*, ι^*) , we have

$$i^* = \sum_{k=1}^{|\vec{F}|} r_k \cdot \vec{i}_F[k] \quad \wedge \quad \iota^* = \sum_{k=1}^{|\vec{F}|} r_k \cdot \vec{\iota}_F[k].$$

where $\mathbf{r}^* = (r_1, \dots, r_{|\vec{F}|})$ is a vector of random scalars. This property naturally extends to the indices for all running instances in the PCD.

Our key observation is that the scalar vector $\mathbf{r}^*$ can serve as a representation of the folded index ι^* , since all index encodings are public and binding. To prove that ι^* is correctly folded, the NARK verifier can take as input the index encoding vector $\vec{\iota}_F$ of $\vec{F}$ and the corresponding representation $\mathbf{r}^*$, instead of a list of commitments $\{C_{M,j}^*\}_{j \in [t]}$. Accordingly, the folding verifier in the PCD only needs to check the correctness of the folded representation $\mathbf{r}^*$.

To verify Equation 6, the NARK verifier computes ι^* as $\langle \mathbf{r}, \vec{\iota}_F \rangle$, and queries the polynomial evaluation of i^* at $(\vec{r}_x, \vec{r}_y)$. The resulting $C_{M,j}^*$ in ι^* are referred to as *virtual commitments* in HyperPlonk [17].

Building on this idea, we design an optimized preprocessing NARK, denoted as NARK_{ccs} . Its security is formalized in the following lemma, and the proof is provided thereafter.

Lemma 3 (Security of Preprocessing NARK). *The preprocessing NARK_{ccs} for a fixed predicate set $\vec{F}$ of bounded size achieves perfect completeness and computational knowledge soundness in the random oracle model.*

Proof. Completeness follows by inspection. For knowledge soundness, suppose a malicious prover $\mathcal{P}^*$ can convince the verifier with non-negligible probability. Then, we build an extractor to output a valid witness for the $(i^*, \mathbf{x}^*)$ output by $\mathcal{P}^*$. Since the verifier accepts, Equation 6 must hold for the witness $\mathbf{w}^*$ output by the prover. By the soundness of sum-check protocols, the CCS constraint in Equation 2 holds with overwhelming probability. To break soundness (*i.e.*, pass the verification with $(i^*, \mathbf{x}^*, \mathbf{w}^*) \notin \mathcal{R}_{\text{CCS}}$), the adversary must either:

1. Find a collision in the commitment, *i.e.*, $C = \text{Commit}(\text{pp}; \tilde{\mathbf{w}})$ for an invalid $\tilde{\mathbf{w}}$.
2. Find a different index i' such that $I(i') = \iota^*$, *i.e.*, for all $j \in [t]$,

$$C_{M,j}^* = \text{Commit}(\text{pp}; \tilde{M}_j') \quad \wedge \quad M_j' \neq M_j^*.$$

Both cases occur with negligible probability due to the commitment scheme's binding property. We thus conclude the proof. $\square$

Breaking the Barrier of Index Folding. Building on the previous discussion, the optimization for the holographic folding scheme in PCD is conceptually straightforward. Specifically, we replace the index encoding ι^* in the folding verification key fvk with the original predicate set encodings $\vec{\iota}$ and a scalar vector

r. During each folding step, instead of folding the matrix commitments in the index encoding, the folding verifier simply folds the scalar vectors over the finite field (note that $\vec{\iota}$ remains unchanged for all PCD nodes). Note that this approach introduces a linear term in $|\vec{F}|$ to the folding verifier's complexity, which is not theoretically optimal. Nevertheless, the predicate number $|\vec{F}|$ is constant in most applications, *e.g.*, RISC-V⁶, and our approach substantially reduces the recursive overhead of PCD in practice by replacing expensive group operations with 1000x smaller field operations. We outline the modifications to NIFS as follows:

- **Folding Indexer:** $\mathcal{I}$ takes as input a CCS index i , an ACCS index i' corresponding to a predicate encoding vector $\vec{\iota}_F$, and a scalar vector r' . It initializes r as a binary vector with a single nonzero entry, and outputs $\text{fpk} := (\text{pp}, i, i')$, $\text{fvk} := (\text{pp}, \vec{\iota}, r, r')$.
- **Folding Prover:** Given fpk , the prover runs the same algorithm as in NIFS. $\mathcal{P}$. In particular, a challenge η is sampled, and the folded index is computed as $i^* = i + \eta \cdot i'$.
- **Folding Verifier:** Given fvk , at step 10 of NIFS. $\mathcal{V}$, the verifier computes $r^* = r + \eta \cdot r'$ instead of directly folding the matrix commitments.

Clearly, given the scalar vector r^* and $\vec{\iota}_F$, the verifier can compute $\iota^* = \langle r^*, \vec{\iota}_F \rangle$ such that $\iota^* = I(i^*)$. We formalize the security of the modified folding scheme in the following lemma. By Theorem 2 and Lemma 3, Lemma 4 follows directly.

Lemma 4 (Security of Modified Holographic Folding Scheme). *Let $\vec{F}$ be a fixed predicate set of bounded size. Assume the underlying commitment scheme satisfies the binding property. Then, the modified holographic folding scheme for CCS achieves perfect completeness and computational knowledge soundness in the random oracle model.*

5.2 Full Construction

Next, we present the detailed construction of our multi-predicate PCD built from the holographic folding scheme and the preprocessing NARK.

Construction 2 (Multi-predicate PCD). *Let NIFS be the non-interactive, non-uniform, holographic multi-folding scheme for CCS relations as proposed in Section 4. Let $(r_\perp, x_\perp, w_\perp)$ be a dummy atomic CCS tuple trivially satisfying any public parameters. According to Definition 12, we can construct a scheme consisting of polynomial-time algorithms $\text{PCD} = (\mathcal{G}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ for a specified set of compliance predicates $\vec{F}$.*

$\text{pp} \leftarrow \mathcal{G}(1^\lambda):$

- 1: Sample pp_{CCS} for $\mathcal{R}_{\text{ACCS}}$ relations.
- 2: Sample $\text{pp}_{\text{fs}} \leftarrow \text{NIFS}.\mathcal{G}(1^\lambda)$ for folding schemes.
- 3: Output $\text{pp} := (\text{pp}_{\text{CCS}}, \text{pp}_{\text{fs}})$.

⁶ <https://github.com/risc0/risc0.git>.

$(\text{ipk}, \text{ivk}) \leftarrow \mathcal{I}(\text{pp}, \vec{F})$:

- 1: Generate indices $\vec{i}$ for the predicate set $\vec{F}$.
- 2: Set $(\text{fpk}, \text{fvk}) \leftarrow \text{NIFS}.\mathcal{I}(\text{pp}_{\text{fs}}, \vec{i}_F)$, where
 $\text{fpk} := \vec{i}_F, \text{fvk} := (\vec{r}_F, \vec{r}_\perp)$.
- 3: Output $\text{ipk} := (\text{fpk}, \vec{F})$, $\text{ivk} = (\text{fvk})$.

We define the recursive circuits R as follows, where fvk is the folding verifier key, $(\vec{x}, \vec{x}')$ are vectors of $\mathcal{R}_{\text{CCS}}$ and $\mathcal{R}_{\text{ACCS}}$ instances. Since R can be computed in polynomial time, it can be represented as a committed CCS index i . Denote the instantiated $\mathcal{R}_{\text{CCS}}$ tuple from the circuit R with inputs as (i, x, w) , where

$$(i, x, w) \leftarrow \text{trace}(R, (\text{fvk}, \vec{x}, \vec{x}', \text{idx}, (z, z_{\text{loc}}, \vec{z}), r^*, x^*, \vec{\pi})).$$

$0/1 \leftarrow R(\text{fvk}, \vec{x}, \vec{x}', \text{idx}, (z, z_{\text{loc}}, \vec{z}), r^*, x^*, \vec{\pi})$:

- 1: Compute $\text{idx}' \leftarrow \varphi(z_{\text{loc}}, \vec{z})$ and check $F_{\text{idx}}(z, z_{\text{loc}}, \vec{z}) = 1$.
- 2: If $z_i = \perp$ for all $z_i \in \vec{z}$, output $\text{Hash}(\text{fvk}, z, r_\perp, x_\perp)$.
- 3: Otherwise parse fvk as $(\vec{r}_F, \vec{r}, \vec{r}')$, and run
 - a: For all $x_i \in \vec{x}$ and $x_i \neq \perp$, check that $x_i.\text{io} = \text{Hash}(\text{fvk}, \text{idx}, z_i, r'_i, x'_i)$
 - b: Run $b \leftarrow \text{NIFS}.\mathcal{V}(\text{fvk}, \vec{x}, \vec{x}', r^*, x^*, \pi)$
 - c: Output $\text{Hash}(\text{fvk}, \text{idx}', z, r^*, x^*)$.
- 4: If the above checks hold, output 1; otherwise, output 0.

$\pi_{\text{pcd}} \leftarrow \mathcal{P}(\text{ipk}, z, z_{\text{loc}}, \vec{z}, \vec{\pi})$:

- 1: Parse $\vec{\pi}$ as a vector of $\mathcal{R}_{\text{CCS}}$ instance-witness pairs $(\vec{x}, \vec{w})$, and a vector of $\mathcal{R}_{\text{ACCS}}$ instance-witness pairs $(\vec{x}', \vec{w}')$.
- 2: Set $\text{idx}' \leftarrow \varphi(z_{\text{loc}}, \vec{z})$.
- 3: If $i = 0$, then set $(i^*, x^*, w^*, \text{pf}) := (i_\perp, x_\perp, w_\perp, \perp)$, else set $(i^*, x^*, w^*, \pi_{\text{fs}}) \leftarrow \text{NIFS}.\mathcal{P}(\text{fpk}, (\vec{x}, \vec{w}), (\vec{x}', \vec{w}'))$.
- 4: Set $r^* = r + \eta \cdot r'$.
- 5: $(i, x, w) \leftarrow \text{trace}(R, (\text{fvk}, \vec{x}, \vec{x}', \text{idx}, (z, z_{\text{loc}}, \vec{z}), r^*, x^*, \vec{\pi}))$.
- 6: Set r such that $\langle r, \vec{i}_F \rangle = i$.
- 7: Output $\pi_{\text{pcd}} := ((r, x, w), (r^*, x^*, w^*), \text{idx}')$.

$0/1 \leftarrow \mathcal{V}(\text{ivk}, z, \pi_{\text{pcd}})$:

- 1: Parse π_{pcd} as $((r, x, w), (r^*, x^*, w^*), \text{idx}')$.
- 2: Check that $x.\text{io} = \text{Hash}(\text{fvk}, \text{idx}', z, r^*, x^*)$.
- 3: Check that $(r, x, w), (r^*, x^*, w^*)$ are satisfying preprocessing NARK tuples.
- 4: If the above checks hold, output 1; otherwise, output 0.

Theorem 3. $\text{PCD} = (\mathcal{G}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ in Construction 2 for a vector of compliance predicates $\vec{F}$ with constant depth in Definition 12 satisfies the perfect completeness, computational knowledge soundness, and statistical zero-knowledge in the random oracle.

The proof of Theorem 3 is presented in Appendix D. We present the evaluation of complexity below. Note that if the PCD can also be proved to satisfy straightline extractability [20], which allows for running unbounded depth.

Complexity. Denote the random oracle for the sum-check protocol as a hash function Hash_{sc} . For simplicity, assume the arity as μ and $m \approx n$.

The *PCD prover recursive cost* at each step contains

- computing the predicate selection function φ ;
- computing the compliance predicate F ;
- computing $O(\mu)$ times hash function Hash ;
- invoking $\text{zk-NIFS}.\mathcal{V}$ with $O(\log m)$ hash functions Hash_{sc} , $O(\mu \cdot (|\vec{F}| + dq))$ $\mathbb{F}$ -ops, and $O(\mu)$ $\mathbb{G}$ -ops.

The *PCD prover native cost* at each step contains

- invoking $\text{zk-NIFS}.\mathcal{P}$ with $O(\log m)$ hash functions Hash_{sc} , and $O(\mu \cdot (N \cdot t + qmd \log^2 d))$ $\mathbb{F}$ -ops and $O(\mu)$ $\mathbb{G}$ -ops;
- computing $O(1)$ times hash function Hash ;
- computing the satisfying committed CCS instance for the execution of $\mathcal{R}$, which is dominated by computing the commitment of witness $\mathbf{w} := z_{\text{loc}}$ with $O(m)$ $\mathbb{G}$ -ops.

The *proof size* of Π consists of an atomic CCS tuple (r', x', w') and a committed CCS tuple (r, x, w) , which has $O(|\vec{F}| + n)$ $\mathbb{F}$ elements. According to previous work [35,34], the proof size can be further reduced to sublinear size by applying a general-purpose *succinct* NARK like Plonk or Spartan. We mention this construction in Section 6.

The *PCD verifier cost* contains

- invoking $\text{zk-NIFS}.\mathcal{V}$ for two tuples with $2 \log m + \log n$ random oracle queries, $O(|\vec{F}| + dq)$ $\mathbb{F}$ -ops and 2 $\mathbb{G}$ -ops;
- checking the commitments on w^* with $O(n)$ $\mathbb{G}$ -ops .
- checking the validity of folded index in $O(\mu \cdot |\vec{F}|)$ $\mathbb{G}$ -ops.

6 KILONOVA: Preprocessing SNARK for Machine Executions

We follow the machine execution model of MuxProof [25], in which the machine description (*i.e.*, instruction set) is encoded as a machine commitment during preprocessing. To prove the execution of a program, a computation commitment representing the sequence of executed instructions is generated on-the-fly, allowing the verifier to efficiently check its correctness against the machine commitment. In this section, we focus on constructing a preprocessing SNARK for the execution trace, omitting aspects such as external memory consistency and control logic functionality that do not constitute novel contributions of our work.

6.1 Sparse Polynomial Commitment Scheme

Following MuxProof [25], the prover can send a machine commitment for each matrix multilinear extension (MLE) $\tilde{M}(\vec{X}, \vec{Y})$, where $\vec{X} \in \mathbb{F}^{\log m}$ and $\vec{Y} \in \mathbb{F}^{\log n}$, allowing the verifier to query these commitments. Without loss of generality, we assume $m = n$, and that $\tilde{M}$ has at most N non-zero points over the Boolean hypercube $\{0, 1\}^{2 \log n}$. However, the size of $\tilde{M}$ is quadratic in n , resulting in an index encoding complexity of $\Omega(n^2)$, which may be prohibitive for offline preprocessing. To address this, we propose an efficient sparse polynomial commitment scheme that enables the verifier to check evaluations on $\tilde{M}$, with prover complexity scaling linearly in N rather than quadratically in n . The general idea is to adopt the approach proposed by Chiesa et al. [21] and extend it to achieve holography for sparse *multivariate* polynomial commitments.

Technically, the prover can reduce the single evaluation claim on the $\Omega(n^2)$ -size polynomial $\tilde{M}$ to three sub-claims involving $\Omega(N)$ -size polynomials $\text{row}, \text{col}, \text{val}$. Specifically, we redefine the multivariate polynomial $\tilde{M}$ over $\vec{X}, \vec{Y} \in \mathbb{F}^{\log n}$, derived from the matrix $M \in \mathbb{F}^{n \times n}$, as

$$\tilde{M}(\vec{X}, \vec{Y}) := \sum_{\kappa \in N} \tilde{e}q(\vec{X}, \text{row}(\text{Bits}(\kappa))) \cdot \tilde{e}q(\vec{Y}, \text{col}(\text{Bits}(\kappa))) \cdot \tilde{\text{val}}(\text{Bits}(\kappa)),$$

where $\text{Bits}(\kappa) \in \{0, 1\}^{\log n}$ denotes the binary representation of κ . Given challenges $\vec{r}_x, \vec{r}_y \in \mathbb{F}^{\log n}$, the prover must convince the verifier of the evaluation claim $\tilde{M}(\vec{r}_x, \vec{r}_y) = \sigma$. This can be reduced, via a $\log N$ -round sum-check protocol, to the following sub-claims:

$$\text{row}(\vec{r}_\kappa) = \sigma_{\text{row}} \wedge \tilde{\text{col}}(\vec{r}_\kappa) = \sigma_{\text{col}} \wedge \tilde{\text{val}}(\vec{r}_\kappa) = \sigma_{\text{val}}.$$

Thus, the verifier only needs to query the evaluations of the committed polynomials $C_{\text{row}}, C_{\text{col}}, C_{\text{val}}$ at $\vec{r}_\kappa$ and verify the sum-check proof accordingly.

6.2 A zkSNARK for Atomic CCS

The above part provides us with a fundamental approach to achieve preprocessing SNARK. In this part, we describe the concrete instantiation for proving the ACCS relation both succinctly and in zero-knowledge. Consider an atomic CCS index-instance pair (i', x') satisfied by a witness w' , where

$$i' = \{M'_j\}_{j \in [t]}, \tag{7}$$

$$x' = (C', u', i\mathbf{o}', \vec{r}'_x, \vec{r}'_y, v'_1, \dots, v'_t, v'_z). \tag{8}$$

Naively, the prover can send the witness w' directly to demonstrate that (i', x', w') is indeed a satisfying ACCS tuple. However, we additionally require the proof system to achieve (i) a succinct proof size and (ii) a zero-knowledge property. The tricky point is that the ACCS instance x' already contains an element $v'_z = \tilde{z}'(\vec{r}'_y)$, which inevitably leaks the information about the witness w' . We solve this by requiring the final node in the PCD to add another trivially holds ACCS tuple in

the multi-folding scheme, let's say (i_r, x_r, w_r) , where i_r belongs to the predefined predicate index set $\vec{i}$. As a result, the witness is masked as $w'' = w' + \eta \cdot w_r$, where η is the challenge for linear combinations. Given that w_r is randomly sampled from $\mathbb{F}^{n-\ell-1}$, it is clear that the evaluation claim $v_z'' = \tilde{z}''(\vec{r}_y)$ does not leak any information about w' . Moreover, we denote the committed CCS tuple (i, x, w) as the representation of the recursive circuit, including the verification for the above multi-folding scheme, where the previous instances x', x_r are encoded in the witness. Briefly, the prover and verifier run the following algorithms:

- The prover takes inputs as (i'', x'', w'') and the tuple (i, x, w) . Then he runs the folding prover algorithm to obtain a new tuple

$$(i^*, x^*, w^*) \leftarrow \text{NIFS}.\mathcal{P}(\text{fpk}, (x'', w''), (x, w)),$$

and sends the following evaluation proofs

$$\begin{aligned}\pi_j^* &\leftarrow \text{PCS.Eval}(\text{pp}, C_{M,j}^*, (\vec{r}_x^*, \vec{r}_y^*), 2 \log n, \tilde{M}_j^*), \forall j \in [t], \\ \pi_z^* &\leftarrow \text{PCS.Eval}(\text{pp}, C_z^*, \vec{r}_y^*, \log n, \tilde{z}^*),\end{aligned}$$

where $C_{M,j}$'s are sparse polynomial commitments.

- The verifier takes inputs as public parameters including $\vec{t}$, index-instance pair (r^*, x^*) and the SNARK proof including $\pi_1^*, \dots, \pi_t^*, \pi_z^*$. He checks the following
 - computes ι^* from $r^*, \vec{t}$.
 - checks the evaluation proof according to ι^* as

$$\begin{aligned}b_0 &\leftarrow \text{PCS.Check}(\text{pp}, C_{M,j}^*, (\vec{r}_x^*, \vec{r}_y^*), 2 \log n, \pi_j^*), \forall j \in [t], \\ b_1 &\leftarrow \text{PCS.Check}(\text{pp}, C_z^*, \vec{r}_y^*, \log n, \pi_z^*).\end{aligned}$$

For the instantiation of PCS, we adopt the HyperKZG scheme [48], which reduces multilinear polynomial commitments and evaluations to univariate KZG commitments, of which the verifier only needs to perform two pairing operations and a logarithmic number of group scalar multiplications, resulting in efficient performance. Moreover, the HyperKZG PCS inherits the aggregatable property of KZG10 [31], which allows us to aggregate the t evaluation proofs $\{\pi_j\}_{j \in [t]}$ into one.

Finally, we highlight the property of our preprocessing zkSNARK in the following theorem:

Lemma 5 (A preprocessing zkSNARK from HyperKZG). *Assuming the HyperKZG satisfying hiding and knowledge soundness, there exists a zkSNARK in the random oracle model for atomic CCS with the following efficiency characteristics, where n denotes the dimensions of R1CS matrices and N denotes the number of non-zero entries in the matrices: The indexer runs in time $O(N)$; The prover runs in time $O(N \log N)$; The proof length is $O(\log n)$; and the verifier runs in time $O(\log n)$.*

Table 2. Theoretical comparisons between multi-predicate recursive arguments. Denote the predicate set as $\vec{F}$, the witness length $|\mathbf{w}| = \ell$, the instance-witness length as n , the number of matrices in each index as t .

	$\mathcal{P}$ native	$\mathcal{P}$ recursive	proof size
SuperNova [32]	$\ell \mathbb{G}$	$O(1) \text{ RO}, 2 \mathbb{G}$	$(\vec{F} \cdot n) \mathbb{F}$
Protostar [13]	$\ell \mathbb{G}, O(tn^2 + \ell) \mathbb{F}$	$O(1) \text{ RO}, (t + 3) \mathbb{G}$	$n \mathbb{F}$
KILONOVA	$\ell \mathbb{G}, O(tn^2 + \ell) \mathbb{F}$	$O(\log \ell) \text{ RO}, 1 \mathbb{G}$	$n \mathbb{F}$

7 Evaluations

7.1 Theoretical Performance

A high-level comparison of features between KILONova and other IVC/PCD schemes is provided in Table 1 in Section 2. In this section, we conduct a detailed theoretical analysis of the performance of existing multi-predicate (non-uniform) recursive argument schemes, as summarized in Table 2, including SuperNova [32], Protostar [13], and our proposed scheme.

In general, our scheme supports high-degree constraint systems, offering expressiveness comparable to Protostar, whereas SuperNova is restricted to R1CS constraints. Moreover, neither Protostar nor SuperNova natively support multi-folding, which limits their potential for parallelism. For a fair comparison, we evaluate the asymptotic performance of all schemes under a unified baseline, setting the arity parameters to $\mu = \nu = 1$ and the constraint degree to $d = 2$.

As illustrated in Table 2, SuperNova achieves optimal performance in both native and recursive prover costs. This efficiency stems from its adoption of an algorithm closely aligned with the uniform folding scheme. However, a notable drawback is that the prover must maintain a list of all active instances, whose length scales linearly with the predicate size $|\vec{F}|$, thereby imposing significant overhead on prover memory and increasing the PCD proof size.

In contrast, the proof sizes of our scheme and Protostar do not include group elements that scale linearly with the number of uninvoked predicates $|\vec{F}|$. Furthermore, compared to Protostar [13], KILONova incurs more random oracle queries within the recursive circuits, but requires t times fewer group elements, resulting in improved scalability and efficiency.

Overall, our scheme achieves a favorable balance between expressiveness, scalability, and prover efficiency, addressing key limitations of prior works and advancing the state-of-the-art in multi-predicate recursive argument systems.

7.2 Implementations and Evaluations

We validate the theoretical results by implementing a proof-of-concept KILONova project in Rust 1.92.0, using the BN256/Grumpkin elliptic curve cycle. Note that this demo PCD only supports R1CS relations because there is no off-the-shelf Rust library for building CCS circuits (left as future work).

Table 3. Proof Size Comparison between different IVC/PCDs. The last column ‘ratio’ is computed by dividing the proof size of SuperNova by that of KILONOVA.

Predicate No.	SuperNova	Protogalaxy	KILONOVA	Ratio
2	35.5	35.0	37.13	$0.96\times$
4	71.0	35.0	37.13	$1.91\times$
8	142.0	35.0	37.13	$3.82\times$
16	284.0	35.0	37.13	$7.65\times$

We highlight that applying R1CS in our evaluation yields only a lower bound on our improvements relative to other schemes because the high-degree case is not considered. We conducted our experiments on a personal laptop running Linux WSL with a 12th-generation i7 CPU at 3.60 GHz and 32GB of RAM. We ran all tests single-threaded. The codes can be referred to the link (<https://github.com/Anonymous-sadh769/kilonova-poc.git>).

In our experimental study, we systematically evaluate the performance of several state-of-the-art PCD (Proof-Carrying Data) schemes, with a particular emphasis on prover time, proof size, and recursive circuit size. The schemes considered include SuperNova, HyperNova [34], Protogalaxy [26], and KILONOVA as constructed in Section 2. The primary experimental results are summarized in Figure 3 of Section 2.4.

In this section, we further investigate the main improvements achieved by our scheme, particularly regarding the PCD proof size and number of constraints, as detailed in Table 4. For all experiments, we fix the following parameters: the CCS relation parameter $t = 3$ (which is equivalent to R1CS), a predicate set of size 2^6 (set to 1 for HyperNova), an example predicate circuit size of 2^{12} , and a witness length of $n = 2^8$. We vary the number of folding input instances (arity) from 2 to 8, and the degree from 2 to 16.

Table 3 presents a comparison of the PCD proof sizes for each scheme under arity scaling from 2 to 16, where arity denotes the number of running instance outputs at the final vertex in the PCD, prior to SNARK compression. It is important to note that HyperNova does not support multi-predicate settings; therefore, it is excluded from this comparison. Our results indicate that both our scheme and Protogalaxy maintain a constant PCD proof size due to the application of index folding, whereas SuperNova’s proof size grows linearly with respect to the predicate set size $|\tilde{F}|$. In practical deployments, the predicate set typically ranges from 2^5 to 2^7 , which can significantly impact SuperNova’s prover efficiency due to the increased proof size. Furthermore, we observe that the proof size of KILONOVA is slightly larger than that of Protogalaxy. This difference arises because KILONOVA employs a relaxed CCS relation, while Protogalaxy directly folds multiple CCS tuples into a single instance, resulting in a higher prover computation cost for Protogalaxy.

For Table 4, it is important to note that SuperNova does not support high-degree relations or multi-folding; thus, we only report the base case as a baseline for comparison. For HyperNova, we restrict our evaluation to the uniform

case, *i.e.*, index-folding is not incorporated into the recursive circuit. Our results demonstrate that, across varying arity, our scheme exhibits an almost steady increase in constraint count compared to HyperNova, which is primarily attributable to the additional round of the sumcheck protocol. Regarding index-folding, the incremental cost introduced by our scheme is negligible relative to Protogalaxy. Furthermore, scaling the degree has only a minor impact on the overall circuit size.

To further demonstrate our improvement on the index-folding cost, we present a cost breakdown of KILONOVA's and Protogalaxy's recursive overhead, respectively under the arity setting as 8, in Figure 4. Clearly, the index-folding cost has been decreased dramatically to 26.8% in KILONOVA, compared to 58.6% in Protogalaxy.

Table 4. Constraint Number Comparison between different IVC/PCDs (“-” denotes “not applicable”).

Arity	Degree	SuperNova	HyperNova	Protogalaxy	KILONOVA
2	2	10413	21474	37484	30298
	4	-	21508	37552	30366
	8	-	21576	37688	30502
	16	-	21712	37960	30774
4	2	-	24424	47748	33376
	4	-	24458	47816	33444
	8	-	24526	47952	33580
	16	-	24662	48224	33852
8	2	-	29812	67764	38892
	4	-	29846	67832	38960
	8	-	29914	67968	39096
	16	-	30050	68240	39368

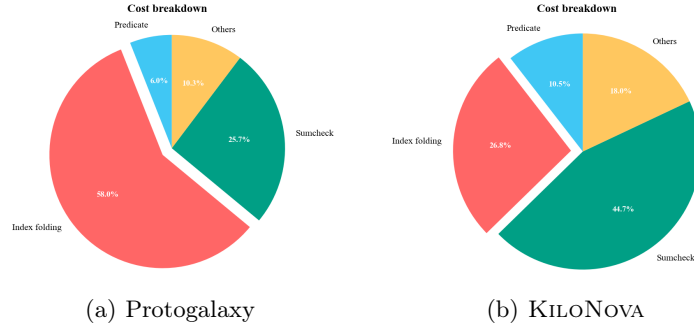


Fig. 4. Recursive overhead breakdown for Protogalaxy and KILONOVA.

References

1. Arun, A., Setty, S.T.V., Thaler, J.: Jolt: Snarks for virtual machines via lookups. In: *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part VI. pp. 3–33. Springer (2024). https://doi.org/10.1007/978-3-031-58751-1_1, https://doi.org/10.1007/978-3-031-58751-1_1
2. Attema, T., Cramer, R.: Compressed-protocol theory and practical application to plug & play secure algorithmics. In: *Annual International Cryptology Conference*. pp. 513–543. Springer (2020), https://doi.org/10.1007/978-3-030-56877-1_18
3. Attema, T., Cramer, R., Kohl, L.: A compressed Σ -protocol theory for lattices. In: *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part II*. pp. 549–579. Springer (2021). https://doi.org/10.1007/978-3-030-84245-1_19, https://doi.org/10.1007/978-3-030-84245-1_19
4. Ben-Sasson, E., Bentov, I., Horeish, Y., Riabzev, M.: Scalable, transparent, and post-quantum secure computational integrity. *Cryptology ePrint Archive* (2018)
5. Ben-Sasson, E., Chiesa, A., Genkin, D., Tromer, E., Virza, M.: Snarks for C: verifying program executions succinctly and in zero knowledge. In: Canetti, R., Garay, J.A. (eds.) *Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part II*. pp. 90–108. Springer (2013). https://doi.org/10.1007/978-3-642-40084-1_6, https://doi.org/10.1007/978-3-642-40084-1_6
6. Ben-Sasson, E., Chiesa, A., Tromer, E., Virza, M.: Scalable zero knowledge via cycles of elliptic curves. *Algorithmica* pp. 1102–1160 (2017). <https://doi.org/10.1007/S00453-016-0221-0>, <https://doi.org/10.1007/s00453-016-0221-0>
7. Bitansky, N., Canetti, R., Chiesa, A., Tromer, E.: From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In: Goldwasser, S. (ed.) *Innovations in Theoretical Computer Science 2012*, Cambridge, MA, USA, January 8-10, 2012. pp. 326–349. ACM (2012). <https://doi.org/10.1145/2090236.2090263>, <https://doi.org/10.1145/2090236.2090263>
8. Bitansky, N., Canetti, R., Chiesa, A., Tromer, E.: Recursive composition and bootstrapping for snarks and proof-carrying data. In: *Proceedings of the forty-fifth annual ACM symposium on Theory of computing*. pp. 111–120 (2013), <https://doi.org/10.1145/2488608.2488623>
9. Boneh, D., Drake, J., Fisch, B., Gabizon, A.: Halo infinite: Proof-carrying data from additive polynomial commitments. In: *Advances in Cryptology-CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part I* 41. pp. 649–680. Springer (2021), https://doi.org/10.1007/978-3-030-84242-0_23
10. Bootle, J., Cerulli, A., Groth, J., Jakobsen, S.K., Maller, M.: Arya: Nearly linear-time zero-knowledge proofs for correct program execution. In: *Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part I*. pp. 595–626. Springer (2018). https://doi.org/10.1007/978-3-030-03326-2_20, https://doi.org/10.1007/978-3-030-03326-2_20
11. Bowe, S., Grigg, J., Hopwood, D.: Recursive proof composition without a trusted setup. *Cryptology ePrint Archive* (2019)

12. Bowe, S., Grigg, J., Hopwood, D.: Halo2. <https://github.com/zcash/halo2> (2020)
13. Bünz, B., Chen, B.: Protostar: Generic efficient accumulation/folding for special sound protocols. Cryptology ePrint Archive (2023), https://doi.org/10.1007/978-981-99-8724-5_3
14. Bünz, B., Chiesa, A., Lin, W., Mishra, P., Spooner, N.: Proof-carrying data without succinct arguments. In: Advances in Cryptology–CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part I 41. pp. 681–710. Springer (2021), https://doi.org/10.1007/978-3-030-84242-0_24
15. Bünz, B., Chiesa, A., Mishra, P., Spooner, N.: Proof-carrying data from accumulation schemes. Cryptology ePrint Archive (2020)
16. Bünz, B., Mishra, P., Nguyen, W., Wang, W.: Arc: Accumulation for reed–solomon codes. In: Annual International Cryptology Conference. pp. 128–160. Springer (2025)
17. Chen, B., Bünz, B., Boneh, D., Zhang, Z.: Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 499–530. Springer (2023)
18. Chiesa, A.: Proof-carrying data. Ph.D. thesis, Massachusetts Institute of Technology (2010)
19. Chiesa, A., Forbes, M.A., Spooner, N.: A zero knowledge sumcheck and its applications. arXiv preprint arXiv:1704.02086 (2017)
20. Chiesa, A., Guan, Z., Samocha, S., Yogev, E.: Security bounds for proof-carrying data from straightline extractors. In: Theory of Cryptography Conference. pp. 464–496. Springer (2024)
21. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, N., Ward, N.: Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In: Advances in Cryptology–EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I 39. pp. 738–768. Springer (2020)
22. Chiesa, A., Ojha, D., Spooner, N.: Fractal: Post-quantum and transparent recursive proofs from holography. In: Advances in Cryptology–EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I 39. pp. 769–793. Springer (2020), https://doi.org/10.1007/978-3-030-45721-1_27
23. Chiesa, A., Tromer, E., Virza, M.: Cluster computing in zero knowledge. In: Advances in Cryptology–EUROCRYPT 2015: 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26–30, 2015, Proceedings, Part II 34. pp. 371–403. Springer (2015)
24. Chu, S., Gomes, B.H., Iglesias, F.H., Norton, T., Tebbs, D.: Uniplonk: Plonk with universal verifier. Cryptology ePrint Archive (2023)
25. Di, Z., Xia, L., Nguyen, W.D., Tyagi, N.: Muxproofs: Succinct arguments for machine computation from vector lookups. In: Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9–13, 2024, Proceedings, Part V. pp. 236–265. Springer (2024). https://doi.org/10.1007/978-981-96-0935-2_8, https://doi.org/10.1007/978-981-96-0935-2_8
26. Eagen, L., Gabizon, A.: Protogalaxy: Efficient protostar-style folding of multiple instances. Cryptology ePrint Archive (2023)

27. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Conference on the theory and application of cryptographic techniques. pp. 186–194. Springer (1986), https://doi.org/10.1007/3-540-47721-7_12
28. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive (2019)
29. Gao, S., Li, G., Fu, H.: ZKWASM: A ZKSNARK WASM emulator. IEEE Trans. Serv. Comput. pp. 4508–4521 (2024). <https://doi.org/10.1109/TSC.2024.3422798>, <https://doi.org/10.1109/TSC.2024.3422798>
30. Groth, J.: On the size of pairing-based non-interactive arguments. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 305–326. Springer (2016)
31. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: Advances in Cryptology-ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings 16. pp. 177–194. Springer (2010), https://doi.org/10.1007/978-3-642-17373-8_11
32. Kothapalli, A., Setty, S.: Supernova: Proving universal machine executions without universal circuits. Cryptology ePrint Archive (2022)
33. Kothapalli, A., Setty, S.: Cyclefold: Folding-scheme-based recursive arguments over a cycle of elliptic curves. Cryptology ePrint Archive (2023)
34. Kothapalli, A., Setty, S.: Hypernova: Recursive arguments for customizable constraint systems. In: Annual International Cryptology Conference. pp. 345–379. Springer (2024), https://doi.org/10.1007/978-3-031-68403-6_11
35. Kothapalli, A., Setty, S., Tzialla, I.: Nova: Recursive zero-knowledge arguments from folding schemes. In: Annual International Cryptology Conference. pp. 359–388. Springer (2022), https://doi.org/10.1007/978-3-031-15985-5_13
36. Lavin, R., Liu, X., Mohanty, H., Norman, L., Zaarour, G., Krishnamachari, B.: A survey on the applications of zero-knowledge proofs. CoRR (2024). <https://doi.org/10.48550/ARXIV.2408.00243>, <https://doi.org/10.48550/arXiv.2408.00243>
37. Liu, T., Zhang, Z., Zhang, Y., Hu, W., Zhang, Y.: Ceno: Non-uniform, segment and parallel zero-knowledge virtual machine. J. Cryptol. p. 17 (2025). <https://doi.org/10.1007/S00145-024-09533-2>, <https://doi.org/10.1007/s00145-024-09533-2>
38. Lund, C., Fortnow, L., Karloff, H., Nisan, N.: Algebraic methods for interactive proof systems. Journal of the ACM (JACM) **39**(4), 859–868 (1992), <https://doi.org/10.1109/fscs.1990.89518>
39. Nguyen, W., Datta, T., Chen, B., Tyagi, N., Boneh, D.: Mangrove: A scalable framework for folding-based snarks. In: Annual International Cryptology Conference. pp. 308–344. Springer (2024)
40. Sanka, A.I., Cheung, R.C.C.: A systematic review of blockchain scalability: Issues, solutions, analysis and future research. J. Netw. Comput. Appl. p. 103232 (2021). <https://doi.org/10.1016/J.JNCA.2021.103232>, <https://doi.org/10.1016/j.jnca.2021.103232>
41. Schwartz, J.T.: Fast probabilistic algorithms for verification of polynomial identities. Journal of the ACM (JACM) **27**(4), 701–717 (1980), <https://doi.org/10.1145/322217.322225>

42. Setty, S.: Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In: Annual International Cryptology Conference. pp. 704–737. Springer (2020), https://doi.org/10.1007/978-3-030-56877-1_25
43. Setty, S., Thaler, J., Wahby, R.: Customizable constraint systems for succinct arguments. Cryptology ePrint Archive (2023)
44. Thaler, J., et al.: Proofs, arguments, and zero-knowledge. Foundations and Trends® in Privacy and Security 4(2–4), 117–660 (2022), <https://doi.org/10.1561/9781638281252>
45. Valiant, P.: Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In: Theory of Cryptography: Fifth Theory of Cryptography Conference, TCC 2008, New York, USA, March 19–21, 2008. Proceedings 5. pp. 1–18. Springer (2008), https://doi.org/10.1007/978-3-540-78524-8_1
46. Xie, T., Zhang, J., Zhang, Y., Papamanthou, C., Song, D.: Libra: Succinct zero-knowledge proofs with optimal prover computation. In: Advances in Cryptology—CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III 39. pp. 733–764. Springer (2019), https://doi.org/10.1007/978-3-030-26954-8_24
47. Zhang, Y., Genkin, D., Katz, J., Papadopoulos, D., Papamanthou, C.: vRAM: Faster verifiable RAM with program-independent preprocessing. In: 2018 IEEE Symposium on Security and Privacy, SP 2018, Proceedings, 21–23 May 2018, San Francisco, California, USA. pp. 908–925 (2018). <https://doi.org/10.1109/SP.2018.00013>, <https://doi.org/10.1109/SP.2018.00013>
48. Zhao, J., Setty, S.T.V., Cui, W., Zaverucha, G.: Micronova: Folding-based arguments with efficient (on-chain) verification. In: IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12–15, 2025. pp. 1964–1982. IEEE (2025). <https://doi.org/10.1109/SP61157.2025.00168>, <https://doi.org/10.1109/SP61157.2025.00168>
49. Zheng, T., Gao, S., Guo, Y., Xiao, B.: Quasar: Sublinear accumulation schemes for multiple instances. Cryptology ePrint Archive (2025)
50. Zhou, Z., Zhang, Z., Dong, J.: Proof-carrying data from multi-folding schemes. IACR Cryptol. ePrint Arch. p. 1282 (2023), <https://eprint.iacr.org/2023/1282>
51. Zhou, Z., Zhang, Z., Dong, J.: Proof-carrying data from multi-folding schemes. Cryptology ePrint Archive (2023)

A Related Works

A.1 SNARKs for Machine Execution

Unrolled proof systems that rely on universal constraint systems incur a computational cost proportional to the size of the universal constraint system for each unrolled instruction (*i.e.*, $O(n|\vec{F}|C)$) as the universal constraints are repeated for every execution step, including Jolt [1] and Ceno [37]. Here we denote n as the number of executed instructions, $|\vec{F}|$ as the size of the instruction set, and C as the constraint number of each instruction. To address this inefficiency, alternative unrolling approaches such as vRAM [47] and muxProof avoid universal constraint systems and achieve prover computation proportional only to the number of executed instructions (*i.e.*, $O((n + |\vec{F}|)C)$). However, unrolled proof systems face inherent limitations in providing zero-knowledge for program

Table 5. Comparisons between existing solutions for universal machine executions.

	CS	Paral.	Recur.	Circuit size	Prover mem.
Ceno [37]	GKR	✓	✗	$O(n \vec{F} C)$	$O(n \vec{F} C)$
Jolt [1]	Plonkish	✗	✗	$O(n \vec{F} C)$	$O(n \vec{F} C)$
MuxProof [25]	Plonkish	✗	✗	$O((n + \vec{F})C)$	$O((n + \vec{F})C)$
SuperNova [32]	R1CS	✗	✓	$O((n + \vec{F})C)$	$O(\vec{F} C)$
Protostar [13]	SPS	✗	✓	$O((n + \vec{F})C)$	$O(C)$
Mangrove [39]	Plonkish	✓	✓	$O((n + \vec{F})C)$	$O(nC)$

Note: we denote $\vec{F}$ as the predicate set, n as the executed instruction number, C as the size of each instruction circuit.

execution as mentioned in [25]. At a minimum, these systems must reveal an upper bound on the number of execution steps.

Incremental proof systems for machine computation have been extensively studied. Ben-Sasson *et al.* [6] demonstrated how proofs for machine execution can be constructed incrementally using recursive proofs. In their approach, each step verifies a single instruction and recursively verifies a SNARK for the previous step, relying on a universal constraint system that encodes the entire instruction set. To address this inefficiency, alternative strategies have been proposed that avoid the universal constraint system, such as SuperNova [32] and Protostar [13]. These innovations eliminate the asymptotic dependence on the instruction set size when opening instruction commitments. As a result, SuperNova and Protostar achieve proofs for machine computation using IVC with a prover cost of $O((n + |\vec{F}|)C)$, formalized as non-uniform IVC. Mangrove [39] can be regarded as a PCD application of Protogalaxy, where the authors proposed a tree for recording and compressing the indecies.

To see the different from our scheme, we highlight that KILONOVA proves in the same CCS language as HyperNova [34] to support high-degree constraints. Other expressive schemes, such as Protostar [13] and Protogalaxy [26], apply special sound protocols (SPS) to support high-degree constraints. In addition, our scheme efficiently supports both non-uniform circuits and multi-folding, which are currently provided only by Protogalaxy [26]. Although a recent work called UniPlonk [24] uniformizes the verifier’s work in Plonk to allow Halo2 [12] and Nova [35] supporting non-uniform circuits, it is unknown whether it can be applied to systems based on multi-folding schemes. For zero-knowledge property, Schemes that consider only one prover and apply zkSNARKs for proving the final IVC/PCD proofs, *e.g.*, Nova [35], are not counted as zero-knowledge IVC/PCD. Therefore, BCLMS21 is the only scheme among those we compared that provides zero-knowledge.

A.2 Folding Schemes

In this work, we mainly compare our PCD’s theoretical performance with BCLMS21 and Protogalaxy. The detailed theoretical performance comparison is given in

Table 1, where $|F|$ is denoted as the size of a predicate function, $|\vec{F}|$ denotes the predicate number.

BCLMS21. Bünz et al. introduce PCD in BCLMS21 [14] from the split accumulation scheme that accumulates the proof of an NARK for uniform R1CS relations. Consider folding s tuples at one step, The recursive overhead (dominated by folding verifier cost) of BCLMS21 is relatively high, requiring 5 MSMs of size n . While KILONOVA only requires 1 MSM. In terms of folding scheme prover native cost, BCLMS21 needs to handle $O(s \cdot |w|)$ group operations with a larger constant term than our scheme. However, it avoids logarithmic random oracle queries. Besides, BCLMS21 also implements zero-knowledge for its accumulation scheme by directly adding masking values. As a result, the prover sends an extra 5G as cross terms for each tuple. For the PCD verifier, since BCLMS21 also allocates $|\vec{F}|$ folded accumulators for different circuits, the verification cost is linearly dependent on the $|\vec{F}|$. Notably, since BCLMS21 does not support d -degree circuits and lookup operations, the above efficiency is analyzed on R1CS relations with 2-degrees.

Protogalaxy. As the follow-up work of Protostar [13], Protogalaxy [26] reduces the cross terms from $O(d^s)$ to $O(ds)$ when folding s non-uniform tuples and requires only $O(1)$ random oracle queries. Though Protogalaxy seems to be a promising primitive for constructing non-uniform PCD, it faces challenges in concrete implementations. First, it needs to handle cross items with Lagrange bases, yielding a *quasi-linear* prover cost scaling with the number of tuples s . While ours is *linear*. To address this problem, Protogalaxy proposes an alternative construction based on sum-check, replacing Lagrange bases with $\tilde{eq}(\cdot)$. However, the sum-check protocol increases the number of RO queries to $O(\log N)$. Besides, its prover's cost is still consistently higher than ours because it computes extra s evaluations of $\tilde{eq}(\cdot)$. Second, Protogalaxy does not support the optimization technique we proposed in Section 5.1, and thus the recursive overhead includes substantial computation for folding the structures. Lastly, the authors of Protogalaxy neither provide constructions for non-uniform PCD nor add zero-knowledge, whereas KILONOVA provides explicit descriptions and addresses potential technical problems.

B Formal Definitions

B.1 Security Definitions of PCS

Completeness. PC has completeness if for all ℓ -variate multilinear polynomial $\tilde{f} \in \mathbb{F}[\ell]$, the following probability is close to 1:

$$\Pr \left[\text{Check}(\text{pp}_{\text{pc}}, C, \vec{x}, y, \ell, \pi) = 1 \wedge f(\vec{x}) = y \mid \begin{array}{l} \text{pp}_{\text{pc}} \leftarrow \text{Setup}(1^\lambda, \ell); \\ C \leftarrow \text{Commit}(\text{pp}, \tilde{f}); \\ \pi \leftarrow \text{Eval}(\text{pp}_{\text{pc}}, C, \vec{x}, y, \ell, \tilde{f}) \end{array} \right].$$

Binding. PC has binding if for any PPT adversary $\mathcal{A}$, size parameter $\ell > 1$, the following probability is negligible:

$$\Pr \left[\begin{array}{l} b_0 = b_1 \neq 0 \\ \wedge \tilde{f}_0 \neq \tilde{f}_1 \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, \ell); \\ (C, \tilde{f}_0, \tilde{f}_1) \leftarrow \mathcal{A}(\text{pp}_{\text{pc}}); \\ b_0 \leftarrow \text{Open}(\text{pp}_{\text{pc}}, C, \tilde{f}_0); \\ b_1 \leftarrow \text{Open}(\text{pp}_{\text{pc}}, C, \tilde{f}_1) \end{array} \right].$$

Knowledge soundness. PC has knowledge soundness if given $\text{pp} \leftarrow \text{Setup}(1^\lambda, \ell)$, $(\text{Eval}, \text{Check})$ is a succinct argument of knowledge for NP relation

$$\mathcal{R}_{\text{Eval}}(\text{pp}) = \left\{ (C, \vec{x}, y; \tilde{f}) : \begin{array}{l} \tilde{f} \in \mathbb{F}[\ell] \wedge \tilde{f}(\vec{x}) = y \\ \wedge \text{Open}(\text{pp}, C, \tilde{f}) = 1 \end{array} \right\}.$$

B.2 Customizable Constraint Systems

The customizable constraint system (CCS) is an intermediate representation of arithmetic circuits introduced by Setty et al. [43], which can simultaneously generalize R1CS, Plonkish, and AIR without overheads.

Definition 8 (CCS [43]). We define the customizable constraint system (CCS) relation $\mathcal{R}_{\text{CCS}}$ as follows. Let the public parameter consist of size bounds $m, n, N, l, t, q, d \in \mathbb{N}$ where $n > l$. An $\mathcal{R}_{\text{CCS}}$ structure $\mathbf{s}$ consists of:

- a sequence of matrices $\{M_j \in \mathbb{F}^{m \times n}\}_{j \in [t]}$ with at most $N = \Omega(\max(m, n))$ non-zero entries in total;
- a sequence of q multisets $\{S_i\}_{i \in [q]}$, where an element in each multiset is from the domain $\{1, \dots, t\}$ and the cardinality of each multiset is at most d .
- a sequence of q constants $\{c_i\}_{i \in [q]}$, where each constant is from $\mathbb{F}$.

A $\mathcal{R}_{\text{CCS}}$ instance consists of public input and output $\mathbf{io} \in \mathbb{F}^l$.

A $\mathcal{R}_{\text{CCS}}$ witness consists of a vector $\mathbf{w} \in \mathbb{F}^{n-l-1}$.

A $\mathcal{R}_{\text{CCS}}$ instance $\mathbf{io}$ with structure $\mathbf{s}$ is satisfied by a $\mathcal{R}_{\text{CCS}}$ witness $\mathbf{w}$ if

$$\sum_{i \in [q]} c_i \cdot \bigcirc_{j \in S_i} M_j \cdot \vec{z} = \vec{0}, \quad (9)$$

where $\vec{z} = (\mathbf{w} || 1 || \mathbf{io}) \in \mathbb{F}^n$, $M_j \cdot \vec{z}$ denotes matrix-vector multiplication, $\bigcirc$ denotes the Hadamard product between vectors, and $\vec{0}$ is an m -sized vector with entries equal to the additive identity in $\mathbb{F}$.

Definition 9 (Committed CCS). Let $\text{PCS} = (\text{Gen}, \text{Com}, \text{Open}, \text{Eval})$ denote an additively-homomorphic polynomial commitment scheme for multilinear polynomials over a finite field $\mathbb{F}$. Denote the public parameters of size bounds as $m, n, N, l, t, q, d \in \mathbb{N}$ where $n = 2 \cdot (l + 1)$ and $\text{pp} \leftarrow \text{Gen}(1^\lambda, s_y)$.

- A $\mathcal{R}_{\text{CCS}}$ structure $\mathbf{s}$ consists of: (i) a sequence of sparse multilinear polynomials in $s_x + s_y$ variables $\{\tilde{M}_j\}_{j \in [t]}$ such that they evaluate to a non-zero value in at most $N = \Omega(m)$ locations over the Boolean hypercube $B_{s_x} \times B_{s_y}$; (ii) a sequence of q multisets $\{S_i\}_{i \in [q]}$, where an element in each multiset is from the domain $\{1, \dots, t\}$ and the cardinality of each multiset is at most d ; (iii) a sequence of q constants $\{c_i\}_{i \in [q]}$, where each constant is from $\mathbb{F}$.

- A $\mathcal{R}_{\text{CCS}}$ instance is (C, io) where C is a commitment to a multilinear polynomial in $s_y - 1$ variables and $\text{io} \in \mathbb{F}^l$.
 - A $\mathcal{R}_{\text{CCS}}$ witness consists of a multilinear polynomial $\tilde{\mathbf{w}}$ in $s_y - 1$ variables.
- A $\mathcal{R}_{\text{CCS}}$ instance with structure $\mathbf{s}$ is satisfied by a witness if $\text{PCS.Com}(\text{pp}, \tilde{\mathbf{w}}) = C$ and if for all $\vec{x} \in B_{s_x}$,

$$\sum_{i \in [q]} c_i \left(\prod_{j \in S_i} \left(\sum_{\vec{y} \in B_{s_y}} \tilde{M}_j(\vec{x}, \vec{y}) \cdot \tilde{z}(\vec{y}) \right) \right) = 0, \quad (10)$$

where $\tilde{z}(\vec{y})$ is an s_y -variate multilinear polynomial such that $\tilde{z}(\vec{y}) = (\mathbf{w}, \tilde{\mathbf{l}}, \text{io})$ for all $\vec{y} \in B_{s_y}$.

B.3 Multi-Predicate Proof-Carrying Data

We extend the definitions of single-predicate PCD in [14] to a non-uniform version with reference to [32]. The derived definitions inherit the theoretical results given in [14]. We start by defining some necessary terminology.

Definition 10. A (distributed computation) transcript $\mathbf{T}$ is a directed acyclic graph with each vertex $v \in V(\mathbf{T})$ labeled by local data $z_{\text{loc}}^{(v)}$ and each edge $e \in E(\mathbf{T})$ labeled by a message $z^{(e)} \neq \perp$. The output $o(\mathbf{T})$ of a transcript $\mathbf{T}$ is a message $z^{(e)}$ where $e = (v, w)$ is the lexicographically-first edge such that w is a sink.

Definition 11. Given a vector of compliance predicates as $\vec{F}$, a vertex $v \in V(\mathbf{T})$ is F -compliant for $F \in \vec{F}$ if for all outgoing edges $e = (v, w) \in E(\mathbf{T})$: (base case) if v has no incoming edges, $F(z^{(e)}, z_{\text{loc}}^{(v)}, \perp, \dots, \perp)$ accepts; (recursive case) if v has incoming edges $e_1, \dots, e_m$, $F(z^{(e)}, z_{\text{loc}}^{(v)}, z^{(e_1)}, \dots, z^{(e_m)})$ accepts. We say that $\mathbf{T}$ is $\vec{F}$ -compliant if each of its vertices is F -compliant with respect to an $F \in \vec{F}$.

Definition 12 (Non-Uniform Proof-Carrying Data). A proof-carrying data scheme with respect to a class of compliance predicates $\mathbf{F}$ is a tuple of algorithms $\text{PCD} = (\mathcal{G}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ where

- $\mathcal{G}(1^\lambda) \rightarrow \text{pp}$ on input security parameter λ , samples and outputs public parameter pp .
- $\mathcal{I}(\text{pp}, \vec{\varphi}) \rightarrow (\text{ipk}, \text{ivk})$ on input public parameter pp , a predicate vector $\vec{\varphi} \subseteq \mathbf{F}$, outputs a prover key ipk and a verifier key ivk .
- $\mathcal{P}(\text{ipk}, z, z_{\text{loc}}, \vec{z}_{\text{in}}, \vec{\pi}_{\text{in}}) \rightarrow \pi$ on input public key ipk , message z of an outgoing edge, local data z_{loc} , messages $\vec{z}_{\text{in}}$ of incoming edges and their corresponding proofs $\vec{\pi}_{\text{in}}$, outputs a new proof π to attest the correctness of z .
- $\mathcal{V}(\text{ivk}, z, \pi) \rightarrow 0/1$ on input verifier key ivk , message z and proof π , outputs 0/1 to reject or accept.

A PCD quadruple $(\mathcal{G}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ should satisfy the perfect completeness, (computational) knowledge soundness, and (statistical) zero knowledge properties as

defined in Appendix B.3. According to [14], such a non-uniform PCD can be constructed from a NARK with its corresponding folding (accumulation) scheme, which is formally stated in Theorem 4.

A multi-predicate proof-carrying data system $\text{PCD} = (\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ should satisfy the perfect completeness, computational knowledge soundness, and statistical zero-knowledge properties as defined below.

Perfect Completeness. PCD has perfect completeness if for every adversary $\mathcal{A}$, the probability $\Pr[A|B]$ equals to 1, where

$$A = \left\{ \begin{array}{c} \text{idx} \leftarrow \varphi(\vec{z}_{\text{in}}, \vec{\pi}_{\text{in}}), F_{\text{idx}}(z, z_{\text{loc}}, \vec{z}_{\text{in}}) \\ \wedge (\forall z_i \in \vec{z}_{\text{in}}, \pi_i \in \vec{\pi}_{\text{in}}, z_i = \perp \vee \mathcal{V}(\text{ivk}, z_i, \pi_i) = 1) \\ \Downarrow \\ \mathcal{V}(\text{ivk}, z, \pi) = 1 \end{array} \right\}.$$

$$B = \left\{ \begin{array}{c} \text{pp} \leftarrow \mathcal{G}(1^\lambda); \\ (\vec{F}, z, z_{\text{loc}}, \vec{z}_{\text{in}}) \leftarrow \mathcal{A}(\text{pp}); \\ (\text{ipk}, \text{ivk}) \leftarrow \mathcal{K}(\text{pp}, \vec{F}); \\ \Pi \leftarrow \mathcal{P}(\text{ipk}, z, z_{\text{loc}}, \vec{z}_{\text{in}}, \vec{\pi}_{\text{in}}) \end{array} \right\}.$$

Knowledge soundness. PCD has knowledge soundness (w.r.t. an auxiliary input distribution $\mathcal{D}$) if for every expected polynomial time adversary $\mathcal{P}^*$, there exists an expected polynomial time extractor $\text{Ext}_{\mathcal{P}^*}$ such that for every set Z , the difference of the following two probabilities is negligible:

$$\Pr \left[\begin{array}{c} \text{idx} \leftarrow \varphi(\vec{z}_{\text{in}}, \vec{\pi}_{\text{in}}) \\ \wedge (\text{pp}, \text{ai}, F_{\text{idx}}, \circ(\mathbf{T}), \text{ao}) \in Z \\ \wedge \mathbf{T} \text{ is } \vec{F}\text{-compliant} \end{array} \middle| \begin{array}{c} \text{pp} \leftarrow \mathcal{G}(1^\lambda); \\ \text{ai} \leftarrow \mathcal{D}(\text{pp}); \\ (F_{\text{idx}}, \mathbf{T}, \text{ao}) \leftarrow \text{Ext}_{\mathcal{P}^*}(\text{pp}, \text{ai}) \end{array} \right],$$

and

$$\Pr \left[\begin{array}{c} \text{idx} \leftarrow \varphi(\vec{z}_{\text{in}}, \vec{\pi}_{\text{in}}) \\ \wedge (\text{pp}, \text{ai}, F_{\text{idx}}, \circ, \text{ao}) \in Z \\ \wedge \mathcal{V}(\text{ivk}, \circ, \pi) = 1 \end{array} \middle| \begin{array}{c} \text{pp} \leftarrow \mathcal{G}(1^\lambda); \\ \text{ai} \leftarrow \mathcal{D}(\text{pp}); \\ (F_{\text{idx}}, \circ, \Pi, \text{ao}) \leftarrow \mathcal{P}^*(\text{pp}, \text{ai}); \\ (\text{ipk}, \text{ivk}) \leftarrow \mathcal{K}(\text{pp}, \varphi, \vec{F}) \end{array} \right].$$

where ai and ao are auxiliary inputs and outputs. These modifications aim to ensure closeness in distribution between the outputs of the prover and the extractor for the strong extraction guarantee [8]. As a result, we can prove post-quantum security for our PCD as mentioned in Theorem 4.

Theorem 4 (Multi-predicate PCD from folding schemes[14]). *Given a NARK for circuit satisfiability and a non-uniform folding scheme for that NARK under the standard model, there exists an efficient transformation that outputs a multi-predicate PCD scheme for constant-depth compliance predicates, assuming that the circuit complexity of the folding verifier is sublinear in its input. Moreover, if the NARK and folding scheme are secure against quantum adversaries, the PCD is also secure against quantum adversaries. If both the NARK and folding scheme are zero-knowledge, the PCD is also zero-knowledge.*

B.4 Security Definitions for Holographic Folding Schemes

We slightly abuse the vector-form denotation $(\vec{i}, \vec{x}, \vec{w}) \in \mathcal{R}^\mu$ to represent that $(\vec{i}[i], \vec{x}[i], \vec{w}[i]) \in \mathcal{R}$ for all $i \in [\mu]$. A public coin interactive holographic folding scheme HFS for $\mathcal{R}, \mathcal{R}'$ satisfies the following requirements.

Perfect Completeness: For all PPT adversaries $\mathcal{A}$, the following probability equals to 1:

$$\Pr \left[\begin{array}{l} \iota^* = I(i^*) \wedge b = 1 \\ \wedge (i^*, x^*, w^*) \in \mathcal{R} \end{array} : \begin{array}{l} \text{pp} \leftarrow \mathcal{G}(1^\lambda) \\ (\vec{i}, \vec{x}, \vec{w}), (\vec{i}', \vec{x}', \vec{w}') \leftarrow \mathcal{A}(\text{pp}) \\ (\vec{i}, \vec{x}, \vec{w}) \in \mathcal{R}^\mu \wedge (\vec{i}', \vec{x}', \vec{w}') \in (\mathcal{R}')^\nu \\ (\text{fpk}, \text{fvk}) \leftarrow \mathcal{I}(\text{pp}, \vec{i}, \vec{i}') \\ (i^*, x^*, w^*, \pi) \leftarrow \mathcal{P}(\text{fpk}, (\vec{x}, \vec{w}), (\vec{x}', \vec{w}')) \\ b \leftarrow \mathcal{V}(\text{fvk}, \vec{x}, \vec{x}', \iota^*, x^*, \pi) \end{array} \right].$$

Knowledge Soundness: For any expected polynomial-time adversaries $\mathcal{A}$ and $\mathcal{P}^*$, the folding scheme is run by $\mathcal{P}^*, \mathcal{V}$, there is an expected polynomial-time extractor Ext such that for all randomness ρ , the following two probabilities are computationally indistinguishable:

$$\Pr \left[\begin{array}{l} (\vec{i}, \vec{x}, \vec{w}) \in \mathcal{R}^\mu \wedge (\vec{i}', \vec{x}', \vec{w}') \in (\mathcal{R}')^\nu : \\ (\vec{i}, \vec{i}'), (\vec{x}, \vec{x}'), \text{st} \leftarrow \mathcal{A}(\text{pp}, \rho) \\ (\vec{w}, \vec{w}') \leftarrow \text{Ext}(\text{pp}, \rho) \end{array} \right]$$

and

$$\Pr \left[\begin{array}{l} \iota^* = I(i^*) \wedge b = 1 \\ \wedge (i^*, x^*, w^*) \in \mathcal{R} : \end{array} \begin{array}{l} \text{pp} \leftarrow \mathcal{G}(1^\lambda), \\ ((\vec{i}, \vec{i}'), (\vec{x}, \vec{x}'), \text{st}) \leftarrow \mathcal{A}(\text{pp}, \rho), \\ (\text{fpk}, \text{fvk}) \leftarrow \mathcal{I}(\text{pp}, \vec{i}, \vec{i}'), \\ (i^*, x^*, w^*, \pi) \leftarrow \mathcal{P}^*(\text{fpk}, (\vec{x}, \vec{w}), (\vec{x}', \vec{w}')) \\ b \leftarrow \mathcal{V}(\text{fvk}, \vec{x}, \vec{x}', \iota^*, x^*, \pi) \end{array} \right].$$

Efficiency: Both communication cost and $\mathcal{V}$'s complexity of the holographic folding scheme should be sublinear in the total size of the original witness as $\circ(\sum_{i \in \mu} |\vec{w}_1[i]| + \sum_{j \in \nu} |\vec{w}_2[j]|)$, i.e., the naive case that the prover directly publishes the entire witness. Additionally, the verifier complexity should also be sublinear in the total size of the indices as $\circ(\sum_{i \in \mu} |\vec{i}_1[i]| + \sum_{j \in \nu} |\vec{i}_2[j]|)$.

Definition 13 (Non-interactive). A holographic folding scheme $(\mathcal{G}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ is non-interactive if the interaction between $\mathcal{P}$ and $\mathcal{V}$ consists of a single message from $\mathcal{P}$ to $\mathcal{V}$. This single message is denoted as $\mathcal{P}$'s output and as $\mathcal{V}$'s input.

Definition 14 (Public coin). A holographic folding scheme $(\mathcal{G}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ is called public coin if all the messages sent from $\mathcal{V}$ to $\mathcal{P}$ are sampled from a uniform distribution.

C Details for Folding Schemes Construction

C.1 Security Proofs for Holographic Folding Scheme

In this section, we present the formal security proofs on the completeness and soundness of holographic folding schemes HFS. The general proving idea is referred to [34] and [19].

Lemma 6. (*Perfect Completeness*). *Construction 1 satisfies perfect completeness.*

Proof. Consider public parameters

$$\text{pp} = (m, n, N, l, t, q, d, \text{pp}_{\text{PCS}}, \text{pp}_{\text{SC}}) \leftarrow \mathcal{G}(1^\lambda)$$

and let $s_x = \log m$ and $s_y = \log n$. Consider arbitrary indices

$$\begin{aligned} i &= (\{\tilde{M}_j\}_{j \in [t]}, \{S_i\}_{i \in [q]}, \{c_i\}_{i \in [q]}) \leftarrow \mathcal{A}(\text{pp}), \\ i' &= (\{\tilde{M}'_j\}_{j \in [t]}, \{S'_i\}_{i \in [q]}, \{c'_i\}_{i \in [q]}) \leftarrow \mathcal{A}(\text{pp}). \end{aligned}$$

Consider prover and verifier key $(\text{fpk}, \text{fvk}) \leftarrow \mathcal{I}(\text{pp}, i, i')$. Suppose the prover and verifier are provided a committed CCS instance

$$\mathbf{x} = (C, \text{io}),$$

and an atomic CCS instance

$$\mathbf{x}' = (C', u', \text{io}', \vec{r}'_x, \vec{r}'_y, \{v'_j\}_{j \in [t]}, v'_z).$$

1. *sum-check protocol#1*: suppose the prover is additionally provided the corresponding satisfying witnesses $\mathbf{w}$ and $\mathbf{w}'$.

For the satisfying committed CCS instance-witness pair, we have, for $\vec{z} = (\mathbf{w}, 1, \text{io})$ and its MLE $\tilde{z}(\vec{y})$, that

$$0 = \sum_{i \in [q]} c_i \cdot \prod_{j \in S_i} \left(\sum_{\vec{y} \in B_{s_y}} \tilde{M}_j(\vec{x}, \vec{y}) \cdot \tilde{z}(\vec{y}) \right), \forall \vec{x} \in B_{s_x}.$$

Treating the right-hand side of the above equation as a polynomial in $\vec{x}$, because it is multilinear and vanishes on all $\vec{x} \in B_{s_x}$, we have that it must be the zero polynomial. Therefore, we have, for $\vec{\alpha}$ sampled by the verifier, that

$$\begin{aligned} 0 &= \sum_{i \in [q]} c_i \cdot \prod_{j \in S_i} \left(\sum_{\vec{y} \in B_{s_y}} \tilde{M}_j(\vec{\alpha}, \vec{y}) \cdot \tilde{z}(\vec{y}) \right) \\ &= \sum_{\vec{x} \in B_{s_x}} \tilde{q}(\vec{\alpha}, \vec{x}) \cdot \left(\sum_{i \in [q]} c_i \prod_{j \in S_i} \left(\sum_{\vec{y} \in B_{s_y}} \tilde{M}_j(\vec{x}, \vec{y}) \cdot \tilde{z}(\vec{y}) \right) \right) \\ &= \sum_{\vec{x} \in B_{s_x}} Q(\vec{x}). \end{aligned}$$

For the satisfying atomic CCS instance-witness pair, we have, for $\vec{z}' = (\mathbf{w}' || \mathbf{u}' || \mathbf{io}')$ and its MLE $\vec{z}'(\vec{y})$, that

$$\begin{aligned} v'_j &= \sum_{\vec{y} \in B_{s_y}} \tilde{e}q(\vec{r}'_y, \vec{y}) \cdot \tilde{M}'_j(\vec{r}'_x, \vec{y}) \\ &= \sum_{\vec{x} \in B_{s_x}} \tilde{e}q(\vec{r}'_x, \vec{x}) \cdot \left(\sum_{\vec{y} \in B_{s_y}} \tilde{e}q(\vec{r}'_y, \vec{y}) \cdot \tilde{M}'_j(\vec{x}, \vec{y}) \right) \\ &= \sum_{\vec{x} \in B_{s_x}} L_j(\vec{x}), \forall j \in [t]. \end{aligned}$$

For γ sampled by the verifier, by linearity, we have that

$$\begin{aligned} \sum_{j \in [t]} \gamma^j \cdot v_j &= \sum_{\vec{x} \in B_{s_x}} \left(\left(\sum_{j \in [t]} \gamma^j \cdot L_j(\vec{x}) \right) + \gamma^{t+1} \cdot Q(\vec{x}) \right) \\ &= \sum_{\vec{x} \in B_{s_x}} f(\vec{x}). \end{aligned}$$

Therefore, by the perfect completeness of the sum-check protocol, we have for $e_1 = \tilde{e}q(\vec{r}'_x, \vec{r}_x^*)$, $e_2 = \tilde{e}q(\vec{\alpha}, \vec{r}_x^*)$ and

$$\begin{aligned} \sigma_j &= \sum_{\vec{y} \in B_{s_y}} \tilde{M}_j(\vec{r}_x^*, \vec{y}) \cdot \tilde{z}(\vec{y}), \forall j \in [t], \\ \sigma'_j &= \sum_{\vec{y} \in B_{s_y}} \tilde{e}q(\vec{r}'_y, \vec{y}) \cdot \tilde{M}'_j(\vec{r}_x^*, \vec{y}), \forall j \in [t], \end{aligned}$$

that

$$\begin{aligned} c_x = f(\vec{r}_x^*) &= \left(\sum_{j \in [t]} \gamma^j \cdot L_j(\vec{r}_x^*) \right) + \gamma^{t+1} \cdot Q(\vec{r}_x^*) \\ &= \left(\sum_{j \in [t]} \gamma^j \cdot e_1 \cdot \sigma'_j \right) + \gamma^{t+1} \cdot e_2 \cdot \sum_{i \in [q]} c_i \cdot \prod_{j \in S_i} \sigma_j. \end{aligned}$$

2. *sum-check protocol*^{#2}: according to the results of *sum-check protocol*^{#1}, for $\tilde{z}(\vec{y})$, $\tilde{z}'(\vec{y})$ and sampled $\vec{r}_x^*$, we have

$$\begin{aligned} \sigma_j &= \sum_{\vec{y} \in B_{s_y}} \tilde{M}_j(\vec{r}_x^*, \vec{y}) \cdot \tilde{z}(\vec{y}) = \sum_{\vec{y} \in B_{s_y}} R_j(\vec{y}), \forall j \in [t], \\ \sigma'_j &= \sum_{\vec{y} \in B_{s_y}} \tilde{e}q(\vec{r}'_y, \vec{y}) \cdot \tilde{M}'_j(\vec{r}_x^*, \vec{y}) = \sum_{\vec{y} \in B_{s_y}} T_j(\vec{y}), \forall j \in [t], \\ v'_z &= \sum_{\vec{y} \in B_{s_y}} \tilde{e}q(\vec{r}'_y, \vec{y}) \cdot \tilde{z}'(\vec{y}) \end{aligned}$$

For δ sampled by the verifier, by linearity, we have that

$$\begin{aligned}
& \sum_{j \in [t]} \delta^j \cdot \sigma_j + \delta^{t+1} \cdot v_z + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot \sigma'_j \\
&= \sum_{j \in [t]} \delta^j \cdot R_j(\vec{y}) + \delta^{t+1} \cdot S(\vec{y}) + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot T_j(\vec{y}) \\
&= \sum_{j \in [t]} g(\vec{y}).
\end{aligned}$$

Therefore, by the perfect completeness of the sum-check protocol, we have for

$$\begin{aligned}
\epsilon &= \tilde{z}(\vec{r}_y^*), \\
\epsilon' &= \tilde{z}'(\vec{r}_y^*), \\
\theta_j &= \tilde{M}_j(\vec{r}_x^*, \vec{r}_y^*), \forall j \in [t], \\
\theta'_j &= \tilde{M}'_j(\vec{r}_x^*, \vec{r}_y^*), \forall j \in [t],
\end{aligned}$$

that

$$\begin{aligned}
c_y &= g(\vec{r}_y^*) \\
&= \sum_{j \in [t]} \delta^j \cdot R_j(\vec{r}_y^*) + \delta^{t+1} \cdot S(\vec{y}) + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot T_j(\vec{r}_y^*) \\
&= \sum_{j \in [t]} \delta^j \cdot e_3 \cdot \theta_j + \delta^{t+1} \cdot \epsilon + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot \theta_j \cdot \epsilon'.
\end{aligned}$$

The above two steps imply that the verifier will not abort. Now, consider the atomic CCS instance obtained from $\mathcal{R}_{\text{CCS}}$ as

$$\mathbf{x} = (C, 1, \text{io}, \vec{r}_x^*, \vec{r}_y^*, \{\theta_j\}_{j \in [t]}, \epsilon).$$

By the precondition that the committed CCS instance (C, io) is satisfied by $\mathbf{w}$ and by the definition of $\{\theta_j\}_{j \in [t]}, \epsilon$, we have that this new atomic CCS instance is also satisfied by $\mathbf{w}$.

Therefore, for random η sampled by the verifier, the folded index $\mathbf{i}^*$ includes $\{M_j^* = M_j + \eta \cdot M'_j\}_{j \in [t]}$, the folded instance $\mathbf{x}^*$ includes $C^* = C + \eta \cdot C'$, $u^* = 1 + \eta \cdot u$, $\text{io}^* = \text{io} + \eta \cdot \text{io}'$, $v_j^* = \theta_j + \eta \cdot \theta'_j$, $v_z^* = \epsilon_j + \eta \cdot \epsilon'_j$. We have that the output folded atomic CCS index-instance pair is satisfied by the witness $\mathbf{w}^* \leftarrow \mathbf{w} + \eta \cdot \mathbf{w}'$ guaranteed by the linearity and the additive homomorphism property of the commitment scheme. $\square$

Lemma 7. (*Knowledge Soundness*). *Construction 1 satisfies knowledge soundness.*

Proof. Consider an adversary $\mathcal{A}$ that adaptively picks the indices and instances, and a malicious prover $\mathcal{P}^*$ that succeeds with probability ϵ . Let $\mathbf{pp} \leftarrow \mathcal{G}(1^\lambda)$. Suppose on input $\mathbf{pp}$ and random tape η , the adversary $\mathcal{A}$ picks two indices

$$\begin{aligned}
\mathbf{i} &= \{\tilde{M}_j\}_{j \in [t]}, \{S_i\}_{i \in [q]}, \{c_i\}_{i \in [q]}, \\
\mathbf{i}' &= \{\tilde{M}'_j\}_{j \in [t]}.
\end{aligned}$$

a new committed CCS instance

$$\mathbf{x} = (C, \mathbf{io}),$$

and an atomic CCS instance

$$\mathbf{x}' = (C', u', \mathbf{io}', \vec{r}_x', \vec{r}_y', \{v_j'\}_{j \in [t]}, v_z'),$$

and some auxiliary state $\mathbf{st}$.

Following the paradigm of the proof for special soundness, we argue that there is a construction of an efficient extractor that outputs the witness $\mathbf{w}, \mathbf{w}'$ with two accepting transcripts from the malicious prover $\mathcal{P}^*$. Concretely, given $\mathbf{w}_1^*, \mathbf{w}_2^*$ from two transcripts under different challenges η_1, η_2 , the extractor simply interpolates the witness $\mathbf{w}, \mathbf{w}'$. The knowledge soundness can be implied by the special soundness according to the theoretical result given in [2].

Next, we must argue that $\mathbf{w}$ and $\mathbf{w}'$ satisfy the remainder of the tuple $(\mathbf{i}, \mathbf{x})$ and $(\mathbf{i}', \mathbf{x}')$ respectively. Consider $\{\theta_j\}_{j \in [t]}, \{\theta_j'\}_{j \in [t]}$ and ϵ, ϵ' sent by the prover, which by the extractor's construction are identical across all executions of the interaction. Then given two different challenges η_1, η_2 in two transcripts, we have that for $i \in \{1, 2\}$ and all $j \in [t]$,

$$v_{j,i}^* = \theta_j + \eta_i \cdot \theta_j', \quad (11)$$

$$v_{z,i}^* = \epsilon + \eta_i \cdot \epsilon'. \quad (12)$$

Since $\tilde{\mathbf{w}}^*$ is a satisfying witness, for $i \in \{1, 2\}$ we have for all $j \in [t]$ that

$$\begin{aligned} v_{j,i}^* &= \tilde{M}_{j,i}^*(\vec{r}_x^*, \vec{r}_y^*), \\ v_{z,i}^* &= \tilde{z}_i^*(\vec{r}_y^*), \end{aligned}$$

where $\tilde{M}_{j,i}^* = \tilde{M}_j + \eta_i \cdot \tilde{M}_j'$, $\tilde{z}_i^* = \widetilde{(\mathbf{w}_i^*, v_{0,i}^*, \mathbf{io}_i^*)} = \tilde{z} + \eta_i \cdot \tilde{z}'$.

Meanwhile, according to equations (11) and (12), for $i \in \{1, 2\}$ and $j \in [t]$,

$$\begin{aligned} \theta_j + \eta_i \cdot \theta_j' &= v_{j,i}^* = \tilde{M}_j(\vec{r}_x^*, \vec{r}_y^*) + \eta_i \cdot \tilde{M}_j'(\vec{r}_x^*, \vec{r}_y^*), \\ \epsilon + \eta_i \cdot \epsilon' &= v_{z,i}^* = \tilde{z}(\vec{r}_y^*) + \eta_i \cdot \tilde{z}'(\vec{r}_y^*), \end{aligned}$$

where $\tilde{z} = \widetilde{(\mathbf{w}, 1, \mathbf{io})}$ and $\tilde{z}' = \widetilde{(\mathbf{w}', u', \mathbf{io}')}$. By interpolating, we have for all $j \in [t]$

$$\begin{aligned} \theta_j &= \tilde{M}_j(\vec{r}_x^*, \vec{r}_y^*), \\ \theta_j' &= \tilde{M}_j'(\vec{r}_x^*, \vec{r}_y^*), \\ \epsilon &= \tilde{z}(\vec{r}_y^*), \\ \epsilon' &= \tilde{z}'(\vec{r}_y^*). \end{aligned}$$

Thus, because the verifier does not abort at step 11, we have c_y as

$$\begin{aligned}
& \sum_{j \in [t]} \delta^j \cdot e_3 \cdot \theta_j + \delta^{t+1} \cdot e_3 \cdot \epsilon + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot \theta'_j \cdot \epsilon' \\
&= \sum_{j \in [t]} \delta^j \cdot \theta_j \cdot \epsilon + \delta^{t+1} \cdot \tilde{eq}(\vec{r}_y, \vec{r}_y^*) \cdot \epsilon' + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot \tilde{eq}(\vec{r}_y', \vec{r}_y^*) \cdot \theta'_j \\
&= \sum_{j \in [t]} \delta^j \cdot \tilde{M}_j(\vec{r}_x^*, \vec{r}_y^*) \cdot \tilde{z}(\vec{r}_y^*) + \delta^{t+1} \cdot \tilde{eq}(\vec{r}_y, \vec{r}_y^*) \cdot \tilde{z}(\vec{r}_y^*) + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot \tilde{eq}(\vec{r}_y, \vec{r}_y^*) \cdot \tilde{M}'_j(\vec{r}_x^*, \vec{r}_y^*) \\
&= \sum_{j \in [t]} \delta^j \cdot R_j(\vec{r}_y^*) + \delta^{t+1} \cdot S(\vec{r}_y^*) + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot T_j(\vec{r}_y^*) \\
&= g(\vec{r}_y^*),
\end{aligned}$$

by the soundness of the sum-check protocol#2, this implies that with probability $1 - O(d \cdot s_y)/|\mathbb{F}| = 1 - \text{negl}(\lambda)$ over the choice of $\vec{r}_y^*$,

$$\begin{aligned}
& \sum_{j \in [t]} \delta^j \cdot \sigma_j + \delta^{t+1} \cdot v_z + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot \sigma'_j \\
&= \sum_{\vec{y} \in B_{s_y}} g(\vec{y}) \\
&= \sum_{\vec{y} \in B_{s_y}} \left(\sum_{j \in [t]} \delta^j \cdot R_j(\vec{y}) + \delta^{t+1} \cdot S(\vec{y}) + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot T_j(\vec{y}) \right) \\
&= \sum_{j \in [t]} \delta^j \cdot \sum_{\vec{y} \in B_{s_y}} R_j(\vec{y}) + \delta^{t+1} \cdot \sum_{\vec{y} \in B_{s_y}} S(\vec{y}) + \delta^{t+1} \cdot \sum_{j \in [t]} \delta^j \cdot \sum_{\vec{y} \in B_{s_y}} T_j(\vec{y}).
\end{aligned}$$

By the Schwartz-Zippel lemma [Sch80], this implies that with probability $1 - O(t)/|\mathbb{F}| = 1 - \text{negl}(\lambda)$ over the choice of δ , for all $j \in [t]$, we have

$$\begin{aligned}
\sigma_j &= \sum_{\vec{y} \in B_{s_y}} R_j(\vec{y}) = \sum_{\vec{y} \in B_{s_y}} \tilde{eq}(\vec{r}_y, \vec{y}) \cdot \tilde{M}_j(\vec{r}_x^*, \vec{y}), \\
v_z &= \sum_{\vec{y} \in B_{s_y}} S(\vec{y}) = \sum_{\vec{y} \in B_{s_y}} \tilde{eq}(\vec{r}_y, \vec{y}) \cdot \tilde{z}(\vec{y}), \\
\sigma'_j &= \sum_{\vec{y} \in B_{s_y}} T_j(\vec{y}) = \sum_{\vec{y} \in B_{s_y}} \tilde{M}'_j(\vec{r}_x^*, \vec{y}) \cdot \tilde{z}'(\vec{y}).
\end{aligned}$$

Thus, because the verifier does not abort at step 5, we have that

$$\begin{aligned}
c_x &= \left(\sum_{j \in [t]} \gamma^j \cdot e_1 \cdot \sigma_j \right) + \left(\gamma^{t+1} \cdot e_2 \cdot \sum_{i \in [q]} c'_i \cdot \prod_{j \in S_i} \sigma_j \right) \\
&= \left(\sum_{j \in [t]} \gamma^j \cdot \tilde{e}q(\vec{r}_x, \vec{r}_x^*) \cdot \sigma_j \right) + \left(\gamma^{t+1} \cdot \tilde{e}q(\vec{\alpha}, \vec{r}_x^*) \cdot \sum_{i \in [q]} c'_i \cdot \prod_{j \in S_i} \theta_j \right) \\
&= \left(\sum_{j \in [t]} \gamma^j \cdot \tilde{e}q(\vec{r}_x, \vec{r}_x^*) \cdot \sum_{\vec{y} \in B_{s_y}} \tilde{e}q(\vec{r}_y, \vec{y}) \cdot \tilde{M}_j(\vec{r}_x^*, \vec{y}) \right) \\
&\quad + \left(\gamma^{t+1} \cdot \tilde{e}q(\vec{\alpha}, \vec{r}_x^*) \cdot \sum_{i \in [q]} c_i \cdot \prod_{j \in S_i} \sum_{\vec{y} \in B_{s_y}} \tilde{M}'_j(\vec{r}_x^*, \vec{y}) \cdot \tilde{z}'(\vec{y}) \right) \\
&= \sum_{j \in [t]} \gamma^j \cdot L_j(\vec{r}_x^*) + \gamma^{t+1} \cdot Q(\vec{r}_x^*) \\
&= f(\vec{r}_x^*),
\end{aligned}$$

by the soundness of the sum-check protocol#1, this implies that with probability $1 - O(d \cdot s_x)/|\mathbb{F}| = 1 - \text{negl}(\lambda)$ over the choice of $\vec{r}_x^*$,

$$\begin{aligned}
&\sum_{j \in [t]} \gamma^j \cdot v_j + \gamma^{t+1} \cdot 0 \\
&= \sum_{\vec{x} \in B_{s_x}} f(x) \\
&= \sum_{\vec{x} \in B_{s_x}} \left(\left(\sum_{j \in [t]} \gamma^j \cdot L_j(\vec{x}) \right) + \gamma^{t+1} \cdot Q(\vec{x}) \right) \\
&= \sum_{\vec{x} \in B_{s_x}} \gamma^j \cdot \left(\sum_{j \in [t]} L_j(\vec{x}) \right) + \gamma^{t+1} \cdot \sum_{\vec{x} \in B_{s_x}} Q(x).
\end{aligned}$$

By the Schwartz-Zippel lemma [Sch80], this implies that with probability $1 - O(t)/|\mathbb{F}| = 1 - \text{negl}(\lambda)$ over the choice of γ , for all $j \in [t]$, we have

$$v_j = \sum_{\vec{x} \in B_{s_x}} L_j(\vec{x}), \text{ and } 0 = \sum_{\vec{x} \in B_{s_x}} Q(x).$$

Therefore,

$$\begin{aligned}
v_j &= \sum_{\vec{x} \in B_{s_x}} L_j(\vec{x}) \\
&= \sum_{\vec{x} \in B_{s_x}} \tilde{e}q(\vec{r}_x^*, \vec{x}) \cdot \left(\sum_{\vec{y} \in B_s} \tilde{e}q(\vec{r}_y^*, \vec{y}) \cdot \tilde{M}_j(\vec{x}, \vec{y}) \right) \\
&= \tilde{M}_j(\vec{r}_x^*, \vec{r}_y^*).
\end{aligned}$$

This implies that $\tilde{w}$ is a satisfying witness to (i, x) . Finally, we have that

$$\begin{aligned}
0 &= \sum_{\vec{x} \in B_{s_x}} Q(x) \\
&= \sum_{\vec{x} \in B_{s_x}} \tilde{e}q(\vec{\alpha}, \vec{x}) \cdot \left(\sum_{i \in [q]} c_i \cdot \prod_{j \in S_i} \left(\sum_{\vec{y} \in B_{s_y}} \tilde{M}_j(\vec{x}, \vec{y}) \cdot \tilde{z}(\vec{y}) \right) \right) \\
&= \sum_{i \in [q]} c_i \cdot \prod_{j \in S_i} \left(\sum_{\vec{y} \in B_{s_y}} \tilde{M}_j(\vec{\alpha}, \vec{y}) \cdot \tilde{z}(\vec{y}) \right).
\end{aligned}$$

By the Schwartz-Zippel lemma, this implies that with probability $1 - s_x/|\mathbb{F}| = 1 - \text{negl}(\lambda)$ over the choice of $\vec{\alpha}$, we have that for all $\vec{x} \in B_{s_x}$

$$0 = \sum_{i \in [q]} c_i \cdot \prod_{j \in S_i} \left(\sum_{\vec{y} \in B_{s_y}} \tilde{M}_j(\vec{x}, \vec{y}) \cdot \tilde{z}(\vec{y}) \right).$$

This implies that $\tilde{w}$ is a satisfying witness to (i, x) . Thus, if the extractor does not abort, it succeeds in producing satisfying witness $\tilde{w}, \tilde{w}'$ with probability $1 - \text{negl}(\lambda)$. $\square$

D Details for PCD

Before introducing the security proofs for PCD in Construction 2, we first present the construction of the modified folding scheme, which replaces the encoded indices with scalars r .

D.1 Security Proofs

We refer to the security proofs of completeness and knowledge soundness to [51].

Completeness. For public parameter pp , consider arbitrary adversarially chosen messages $(F, z, z_{\text{loc}}, \vec{z}, \vec{\pi})$ satisfying

$$\begin{aligned}
F &\in \vec{F}; F(z, z_{\text{loc}}, \vec{z}) = 1; \\
\forall z_i \in \vec{z}, z_i &= \perp \text{ or } \mathcal{V}(\text{vk}, z_i, \pi_i) = 1,
\end{aligned}$$

such that the perfect completeness precondition is satisfied. We show that given $\pi_{\text{pcd}} \leftarrow \mathcal{P}(\text{ipk}, z, z_{\text{loc}}, \vec{z}, \vec{\pi})$, the verifier algorithm passes, *i.e.*, $\mathcal{V}(\text{ivk}, z, \pi_{\text{pcd}}) = 1$ with probability 1. Specifically, there are two cases:

- If $z_i = \perp$ for all $i \in [\mu]$, the prover runs the algorithm honestly, and the compliance $F(z, z_{\text{loc}}, \vec{z})$ holds by the preconditions. The circuit R can be constructed accordingly as a satisfied $\mathcal{R}_{\text{CCS}}$ tuple

$$(i, x, w) \leftarrow \text{trace}(R, (\text{fvk}, \vec{x}, \vec{x}', \text{idx}, (z, z_{\text{loc}}, \vec{z}), r^*, x^*, \vec{\pi})).$$

As a result, $\mathcal{V}(\text{vk}, z, \pi_{\text{pcd}}) = 1$ with probability 1.

- If $\exists i \in [\mu]$ such that $z_i \neq \perp$, by the perfect completeness precondition, $\{x'_i, w'_i\}_{i \in [\mu]}$ are satisfied $\mathcal{R}_{\text{ACCS}}$ instance-witness pairs, $\{x_i, w_i\}_{i \in [\mu]}$ are satisfied $\mathcal{R}_{\text{CCS}}$ instance-witness pairs, and $x_i.\text{io} = H(\text{fvk}, \text{idx}, z_i, \vec{r}_i, \vec{x}'_i)$. The prover runs the algorithm honestly, and the compliance $F(z, z_{\text{loc}}, \vec{z})$ holds by the preconditions. Then the prover runs generic folding scheme for $(\vec{x}, \vec{w}), (\vec{x}', \vec{w}')$. The circuit R can be constructed accordingly and a satisfied $\mathcal{R}_{\text{CCS}}$ tuple is as

$$(i, x, w) \leftarrow \text{trace}(R, (\text{fvk}, \vec{x}, \vec{x}', \text{idx}, (z, z_{\text{loc}}, \vec{z}), r^*, x^*, \vec{\pi})).$$

By the perfect completeness of the generic folding scheme, we have that (x, w) is a satisfied $\mathcal{R}_{\text{CCS}}$ instance-witness pair. As a result, $\mathcal{V}(\text{vk}, z, \pi_{\text{pcd}}) = 1$ with probability 1.

In conclusion, we show that Construction 2 has perfect completeness.

Knowledge Soundness.

Proof. Given a fixed set Z , $\text{pp} \leftarrow \mathcal{G}(1^\lambda)$ and auxiliary input $\text{ai} \leftarrow \mathcal{D}(\text{pp})$, the polynomial time adversary $\mathcal{P}^*$ succeeds in producing valid transcript $(F_{\text{idx}}, \circ, \Pi, \text{ao})$ with non-negligible probability ϵ . We aim to show that it is feasible to construct an extractor $\text{Ext}_{\mathcal{P}^*}$ on input (pp, ai) , succeeds in outputting $(F_{\text{idx}}, \text{T}, \text{ao})$ with probability $\epsilon - \text{negl}(\lambda)$, where $F_{\text{idx}} \in \mathcal{F}$, $(\text{pp}, \text{ai}, F_{\text{idx}}, \text{ao}) \in Z$ and T is $\vec{F}$ -compliant. According to [14], it is convenient to assume the transcript T as a d -depth tree. Among the tree T , each node u with local data $z_{\text{loc}}^{(u)}$ has a unique outgoing edge labelled with $z^{(u)}$ and a proof $\pi_{\text{pcd}}^{(u)}$ for the correctness of $z^{(u)}$. The extractor $\text{Ext}_{\mathcal{P}^*}$ is constructed inductively by constructing a sequence of extractors $\text{Ext}_0, \dots, \text{Ext}_d$. For $i \in 0, \dots, d$, Ext_i outputs a $(i+1)$ -depth tree T_i . Basically, we define $\text{Ext}_0(\text{pp}, \text{ai})$ runs $(F, \circ, \Pi, \text{ao}) \leftarrow \mathcal{P}^*(\text{pp}, \text{ai})$ and outputs $(F, \text{T}_0, \text{ao})$, where T_0 contains only one node labeled with $(\circ, \Pi)$. Then assume we already have extractor Ext_{i-1} . To construct Ext_i , an adversary $\mathcal{P}_{i-1}^*$ for the zero-knowledge non-interactive generic folding scheme needs to be constructed first.

$\mathcal{P}_{i-1}^*(\text{pp}, \text{ai}, \rho)$:

-
- 1 : Compute $(F_{\text{idx}}, \text{T}_{i-1}, \text{ao}) \leftarrow \text{Ext}_{i-1}(\text{pp}, \text{ai})$. If T_{i-1} is not a tree of depth i , abort.
 - 2 : For each node $u \in L_{\text{T}_{i-1}}(i)$, denote its label as $(z^{(u)}, \pi_{\text{pcd}}^{(u)})$.
 - 3 : Parse $\pi_{\text{pcd}}^{(u)}$ as $((x^{(u)}, w^{(u)}), (x'^{(u)}, w'^{(u)}))$.
 - 4 : Obtain $(\{x_i^{(u)}, x'_i{}^{(u)}, z_i^{(u)}\}_{i \in [\mu]}, \pi_{\text{fs}}^{(u)})$ from $w^{(u)}$.
 - 5 : Let $L_{i-1} := \{u \in L_{\text{T}_{i-1}}(i) \mid \exists i \in [\mu], z_i^{(u)} \neq \perp\}$.
 - 6 : Output

$$\left(\left\{ \{x_i^{(u)}, x'_i{}^{(u)}\}_{i \in [\mu]}, x^{(u)}, w^{(u)}, \pi_{\text{fs}}^{(u)} \right\}_{u \in L_{i-1}}, (F_{\text{idx}}, \text{T}_{i-1}, \text{ao}) \right).$$

where $L_{\text{T}_{i-1}}(i)$ denotes the set of nodes of T at depth i . According to the knowledge soundness of the generic folding scheme, we can construct another extractor $\text{Ext}_{\mathcal{P}_{i-1}^*}$. On input $v \in L_{i-1}$, $\text{Ext}_{\mathcal{P}_{i-1}^*}$ outputs $\{w_i^{(u)}, w'_i{}^{(u)}\}_{i \in [\mu]}$ with

non-negligible probability, where $\{x_i^{(u)}, w_i^{(u)}\}_{i \in [\mu]}$ are satisfied committed CCS instance-witness pairs and $\{x_i'^{(u)}, w_i'^{(u)}\}_{i \in [\mu]}$ are satisfied atomic CCS instance-witness pairs.

Based on $\mathcal{P}_{i-1}^*, \text{Ext}_{\mathcal{P}_{i-1}^*}$, we can further construct Ext_i as follows.

$(F_{\text{idx}}, T_i, \text{ao}) \leftarrow \text{Ext}_i(\text{pp}, \text{ai}):$

1 : Run $\leftarrow \text{Ext}_{\mathcal{P}_{i-1}^*}(\text{pp}, \text{ai}, \rho)$ to obtain

$$\left(\left\{ \{x_i^{(u)}, w_i'^{(u)}\}_{i \in [\mu]}, \{x_i'^{(u)}, w_i^{(u)}\}_{i \in [\mu]} \right\}_{u \in L_{i-1}}, (F_{\text{idx}}, T_{i-1}, \text{ao}) \right)$$

If T_{i-1} is not a tree of depth i , abort.

2 : Retrieve $\{w^{(u)}\}_{u \in L_{T_{i-1}}(i)}$ from the internal state of $\mathcal{P}_{i-1}^*$ and obtain $z_{\text{loc}}^{(u)}, \{z_i^{(u)}\}_{i \in [\mu]}$ from $w^{(u)}$.

3 : Append $z_{\text{loc}}^{(u)}$ to the label of $u \in L_{T_{i-1}}(i)$.

4 : For each node $u \in L_{i-1}$, let $L_u := \{i \in [\mu] \mid z_i^{(u)} \neq \perp\}$.

5 : Construct T_i of depth $i+1$ from T_{i-1} by adding, for each node $u \in L_{i-1}$, $(z_i^{(u)}, \Pi_i^{(u)})$ to the label of its child $i \in L_u$, where

$$\Pi_i^{(u)} = \left((x_i^{(u)}, w_i^{(u)}), (x_i'^{(u)}, w_i'^{(u)}) \right).$$

6 : Output $(F_{\text{idx}}, T_i, \text{ao})$.

We claim that for $i \in \{0, 1, \dots, d\}$, the extractor $\text{Ext}_i(\text{pp}, \text{ai})$ outputs $(F_{\text{idx}}, T_i, \text{ao})$ in expected polynomial time such that with probability $\epsilon - \text{negl}(\lambda)$, the following conditions hold

- $F_{\text{idx}} \in \vec{F}$, $(\text{pp}, \text{ai}, F, \circ(T_i), \text{ao}) \in Z$;
- T_i is $\vec{F}$ -compliant up to depth i ;
- for all $u \in L_{T_i}(i+1)$, $\mathcal{V}(\text{vk}, z^{(u)}, \pi_{\text{pcd}}^{(u)}) = 1$.

The correctness of the above claim can be proved by induction.

- (Base case.) Since $\text{Ext}_0(\text{pp}, \text{ai})$ runs $(F_{\text{idx}}, \circ, \Pi, \text{ao}) \leftarrow \mathcal{P}^*(\text{pp}, \text{ai})$, it satisfies the conditions above.
- (Inductive hypothesis.) Assume that the extractor Ext_{i-1} satisfies the above-mentioned conditions.
- (Inductive step.) Based on the hypothesis, we show that Ext_i also satisfies the conditions by the following discussion.

The inductive hypothesis ensures that Ext_{i-1} satisfies with probability $\epsilon - \text{negl}(\lambda)$, that $F_{\text{idx}} \in \vec{F}$, $(\text{pp}, \text{ai}, F_{\text{idx}}, \circ(T_{i-1}), \text{ao}) \in Z$, T_{i-1} is $\vec{F}$ -compliant up to the depth $i-1$, and for all $u \in L_{T_{i-1}}(i)$, $\mathcal{V}(\text{vk}, z^{(u)}, \pi_{\text{pcd}}^{(u)}) = 1$. By the correctness of algorithm $\mathcal{V}$, we have

- (1) $\{(x^{(u)}, w^{(u)}), (x'^{(u)}, w'^{(u)})\}_{u \in L_{T_{i-1}}(i)}$ are satisfied instance-witness pairs.

Since T_{i-1} is $\vec{F}$ -compliant, by the construction of $\mathcal{R}$ and Hash , we have

- (2) for $u \in L_{T_{i-1}}(i)$, $F_{\text{idx}}(z^{(u)}, z_{\text{loc}}^{(u)}, \tilde{z}^{(u)})$ accepts;
- (3) for $u \in L_{i-1}$, $x^{(u)} = \text{zk-NIFS}.\mathcal{V}'(\text{vk}', \{x_i^{(u)}\}_{i \in [\mu]}, \{x_i'^{(u)}\}_{i=0}^\mu, \pi_{\text{fs}}^{(u)})$;

(4) for $u \in L_{i-1}$, $\mathbf{x}_i^{(u)}.io = \text{Hash}(\mathbf{vk}', z_i^{(u)}, \mathbf{x}_i^{(u)}) \forall i \in [\mu]$.

Concretely, (2) implies that T_i is $\vec{F}$ -compliant up to depth i and $F_{\text{idx}} \in \vec{F}$, $(\mathbf{pp}, \mathbf{ai}, F_{\text{idx}}, \circ(\mathsf{T}_i), \mathbf{ao}) \in Z$. (1) and (3) imply that there exists an efficient construction of $\mathcal{P}_{i-1}^*$ that succeeds in producing folded pairs $\{\mathbf{x}^{(u)}, \mathbf{w}^{(u)}\}_{u \in L_{i-1}}$ with probability $\epsilon - \text{negl}(\lambda)$. Then there exists an efficient extractor $\text{Ext}_{\mathcal{P}_{i-1}^*}$ outputting $\{\{\mathbf{w}_i^{(u)}\}_{i \in [\mu]}, \{\mathbf{w}'_i{}^{(u)}\}_{i \in [\mu]}\}_{u \in L_{i-1}}$ guaranteed by the knowledge soundness of generic foldings scheme. (1)-(4) imply that $\mathcal{V}(\mathbf{vk}, z^{(u)}, \pi_{\text{pcd}}^{(u)}) = 1$ holds for all $u \in L_{(T)_i}(i+1)$. Therefore, the hypothesis for Ext_i also holds.

In conclusion, we prove that Construction 2 is knowledge-sound. $\square$